Experiment No.—1: Preparing the IoT Hardware

Aim:

a) Set up Raspberry Pi OS / Arduino IDE
b) Configure GPIO settings and test basic connectivity
c) Demonstrate pin layout and onboard peripherals

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card (16GB or higher)
- Official Raspberry Pi 4 power adapter
- HDMI cable and monitor
- USB keyboard and mouse
- Ethernet cable / Wi-Fi connection (for internet access)

Software:

- Raspberry Pi OS (latest stable version)
- Raspberry Pi Imager

Programming Language: Python

Compiler/IDE: Thonny IDE (Python IDE available in Raspberry Pi OS)

Learning Objectives:

1. Understand the architecture and features of Raspberry Pi 4 Model B used in IoT applications.
2. Install and configure Raspberry Pi OS on the hardware platform.
3. Access and configure GPIO settings using system tools and Python libraries.
4. Identify and interpret the 40-pin GPIO layout and onboard peripherals.
5. Verify basic system connectivity and readiness for hardware interfacing experiments

Theory:

1. Introduction to IoT Hardware Platform

The Internet of Things (IoT) requires a hardware platform capable of processing data, running programs, and interfacing with external devices. A commonly used platform for IoT development is the **Raspberry Pi 4 Model B**, which is a single-board computer.

Unlike microcontrollers, Raspberry Pi:

- Runs a full operating system
- Supports high-level programming languages such as Python
- Provides networking capabilities
- Offers GPIO pins for hardware interfacing

In this experiment, the focus is on preparing the hardware environment before starting device interfacing experiments.

2. Overview of Raspberry Pi 4 Model B

Raspberry Pi 4 Model B is a compact computing board designed for embedded and IoT applications.

Key Features:

- Quad-core processor
- RAM (2GB/4GB/8GB variants)
- Dual HDMI ports
- USB 2.0 and USB 3.0 ports
- Ethernet port
- Built-in Wi-Fi and Bluetooth
- 40-pin GPIO header

These onboard peripherals eliminate the need for additional modules in basic IoT development.

3. Block-Level Representation

The Raspberry Pi system can be conceptually represented as:

```
+-----------------------------------------------------+
|                Raspberry Pi 4 Model B               |
|-----------------------------------------------------|
|  Processor  |  RAM  |  Wi-Fi / Bluetooth            |
|-----------------------------------------------------|
|  USB Ports  |  HDMI |  Ethernet                     |
|-----------------------------------------------------|
|               40-Pin GPIO Header                    |
+-----------------------------------------------------+
```

This structure shows how the processor interacts with memory, communication modules, and GPIO pins.

4. Raspberry Pi OS

Raspberry Pi OS is a Linux-based operating system designed specifically for Raspberry Pi hardware.

Purpose of Installing the OS:

- Enables graphical desktop interface
- Provides terminal access for command execution
- Supports Python programming
- Allows installation of development tools
- Enables GPIO access via software libraries

Without the operating system, the Raspberry Pi cannot function as a programmable IoT platform.

5. GPIO (General Purpose Input/Output)

GPIO pins allow Raspberry Pi to interact with external hardware components.

Each GPIO pin can be configured as:

- Input → Reads signals
- Output → Sends signals

In this experiment, we focus on:

- Accessing GPIO settings
- Understanding pin configuration
- Testing basic GPIO functionality

GPIO operates at **3.3V logic level**, which means:

- HIGH → 3.3V
- LOW → 0V

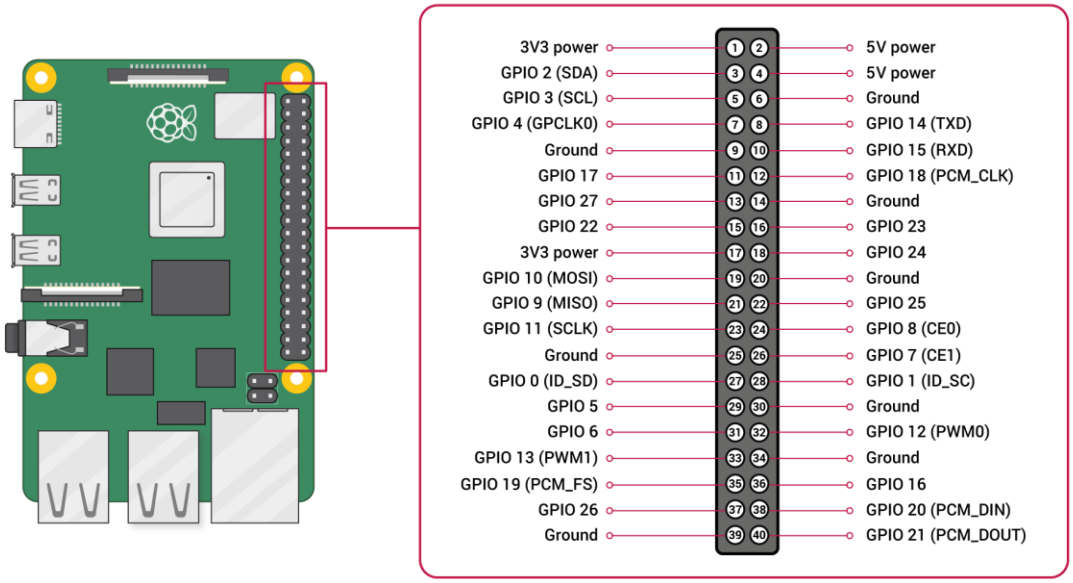
Proper understanding of voltage levels is important to avoid hardware damage.

6. 40-Pin GPIO Layout

Raspberry Pi 4 has a 40-pin header arranged in two rows of 20 pins each.

Simplified GPIO Layout

```
(Physical Pin Numbers)
```



Types of Pins

Pin Type	Function
3.3V / 5V	Power supply
GND	Ground
GPIO	Digital Input/Output
I2C, SPI, UART	Communication pins

Understanding this layout is essential before connecting any external device.

7. GPIO Numbering Systems

Raspberry Pi supports two numbering modes:

Mode	Description
BOARD	Uses physical pin numbers (1–40)
BCM	Uses Broadcom GPIO numbers (GPIO17, GPIO27, etc.)

Example:

- Physical Pin 11 corresponds to GPIO17 (BCM mode).

Choosing the correct numbering mode ensures proper pin configuration in Python programs.

8. Basic Connectivity Verification

Before starting hardware interfacing experiments, it is important to verify:

- The system boots correctly
- The OS is functioning
- Terminal commands execute properly
- Python environment is available
- GPIO library can be imported without errors

A simple Python test such as importing the GPIO library confirms readiness of the development environment.

9. Importance of This Experiment

This experiment lays the foundation for all future IoT practicals because:

- It ensures the hardware platform is correctly configured.
- It prevents software and system-level errors in later experiments.
- It provides clear understanding of GPIO structure and onboard peripherals.

Proper hardware preparation ensures smooth implementation of sensor interfacing, actuator control, and network-based IoT applications in subsequent experiments.

Steps:

1. Preparation of MicroSD Card

1. Insert the microSD card into the computer.
2. Open **Raspberry Pi Imager**.
3. Select:
   - Device: Raspberry Pi 4
   - Operating System: Raspberry Pi OS
   - Storage: Inserted microSD card
4. Click **Write** to flash the OS onto the microSD card.
5. After completion, safely remove the SD card.

2. Booting Raspberry Pi 4 Model B

1. Insert the flashed microSD card into the Raspberry Pi.
2. Connect HDMI cable, keyboard, mouse, and power supply.
3. Power ON the board.
4. Complete the initial setup wizard:
   - Select country, language, and time zone
   - Connect to Wi-Fi or Ethernet
   - Set username and password

3. Updating the System

Open the Terminal and execute:

```
sudo apt update
sudo apt upgrade -y
```

Purpose:

- Updates the package list
- Installs latest security and system updates

4. Accessing Raspberry Pi Configuration

To open configuration settings:

```
sudo raspi-config
```

In the configuration tool:

- Navigate to **System Options** to verify hostname and boot settings
- Navigate to **Interface Options** to enable required interfaces if needed
- Finish and exit

If prompted, reboot the system:

```
sudo reboot
```

5. Verifying Python Installation

Check Python version:

```
python3 --version
```

This confirms that Python is installed and ready for development.

6. Verifying GPIO Library Availability

Open Python (Terminal or Thonny) and test:

```
import RPi.GPIO as GPIO
print("GPIO Library Loaded Successfully")
GPIO.cleanup()
```

If no error appears, GPIO is configured properly.

7. Observing GPIO Pin Layout and Onboard Peripherals

- Identify the 40-pin GPIO header on the board.
- Observe the placement of:
  - Power pins (3.3V, 5V)
  - Ground pins
  - GPIO pins
- Identify onboard peripherals:
  - USB ports
  - HDMI ports
  - Ethernet port
  - Wi-Fi and Bluetooth module

This confirms familiarity with hardware layout before beginning interfacing experiments.

Learning Outcomes:

1. Successfully set up Raspberry Pi OS on Raspberry Pi 4 Model B.
2. Configure system settings and access GPIO functionality without errors.
3. Demonstrate understanding of pin layout and different types of GPIO pins.
4. Confirm proper booting, network connectivity, and Python environment setup.
5. Prepare the IoT hardware platform for future sensor and actuator interfacing experiments.

Course Outcomes:

This experiment supports the attainment of course outcomes by enabling students to set up and configure an IoT hardware platform using Raspberry Pi 4 Model B. By installing Raspberry Pi OS, accessing system configuration tools, and understanding GPIO architecture and onboard peripherals, students develop foundational skills in embedded Linux environments and hardware-software integration. The experiment prepares learners to design, implement, and troubleshoot IoT-based systems by ensuring proper system initialization, connectivity verification, and readiness for sensor and actuator interfacing in subsequent practical applications.

Conclusion:

In this experiment, the Raspberry Pi 4 Model B was successfully prepared as an IoT development platform by installing Raspberry Pi OS and configuring the system environment. Basic system connectivity, Python installation, and GPIO library functionality were verified. The experiment also helped in understanding the GPIO pin layout and onboard peripherals of the Raspberry Pi, which form the foundation for further hardware interfacing experiments in IoT applications.

Viva Questions:

1. What is Raspberry Pi 4 Model B, and why is it suitable for IoT applications?
2. What is the role of Raspberry Pi OS in IoT hardware preparation?
3. What are GPIO pins, and why must they be configured properly before use?
4. What is the difference between BOARD and BCM GPIO numbering modes?
5. Why is it important to update the system and verify Python and GPIO libraries before starting IoT experiments?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—2: GPIO – Light the LED (with and without Button)

Aim:

a) Blink LED using Python (Raspberry Pi) or C++ (Arduino)
b) Add a push button to toggle LED ON/OFF

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- Breadboard
- LED
- Push Button
- Jumper wires

Software:

- Raspberry Pi OS
- RPi.GPIO library

Programming Language: Python

Compiler/IDE: Thonny IDE

Learning Objectives:

1. Understand the working principle of digital input and digital output using GPIO pins.
2. Configure a GPIO pin as OUTPUT to control an LED using Python.
3. Configure a GPIO pin as INPUT to read the state of a push button.
4. Implement time delay and logical conditions in Python for hardware control.
5. Develop basic understanding of interaction between input devices and output devices in embedded systems.

Theory:

1. Introduction to GPIO-Based LED Control

In embedded and IoT systems, hardware components such as LEDs, sensors, and switches communicate with the controller through **GPIO (General Purpose Input/Output)** pins. GPIO pins act as programmable digital pins that can either:

- Send signals (Output mode)
- Receive signals (Input mode)

In this experiment, two fundamental operations are performed:

1. Blinking an LED using GPIO output
2. Using a push button as input to toggle the LED

These operations demonstrate the basic interaction between software and hardware.

2. Digital Output: Controlling an LED

An LED (Light Emitting Diode) is a semiconductor device that emits light when current flows through it.

Working Principle

- GPIO HIGH (3.3V) → Current flows → LED turns ON
- GPIO LOW (0V) → No current → LED turns OFF

Raspberry Pi GPIO pins operate at **3.3V logic level**.

LED Connection Concept

```
GPIO17 (Pin 11) ----->|---- GND
                       LED
```

- Anode (long leg) → GPIO pin
- Cathode (short leg) → Ground

LED Blinking Concept

Blinking is achieved by alternating the GPIO state with a delay.

Logical sequence:

```
Set GPIO as OUTPUT
        ↓
Set HIGH  → LED ON
Delay
Set LOW   → LED OFF
Delay
Repeat
```

The delay is created using time control functions in Python. This demonstrates timed digital signal generation.

3. Digital Input: Push Button Interfacing

A push button is a digital input device that provides two states:

- Pressed
- Released

When connected to a GPIO pin configured as INPUT, the Raspberry Pi reads its state as:

- HIGH (1)
- LOW (0)

Button Connection Concept

```
3.3V ---- Button ---- GPIO27
```

When the button is pressed, the circuit is completed and the GPIO pin reads HIGH.

4. Pull-Up and Pull-Down Concept

If an input pin is left unconnected, it may read random values due to electrical noise. To avoid this, pull resistors are used.

There are two types:

Type	Default State	When Button Pressed
Pull-Down	LOW	HIGH
Pull-Up	HIGH	LOW

In this experiment, internal pull-down configuration is used to ensure stable input reading.

5. LED Toggle Using Push Button

Toggle means changing the LED state every time the button is pressed.

For example:

- First press → LED ON
- Second press → LED OFF
- Third press → LED ON

This requires logical programming using:

- Conditional statements (if condition)
- State variables
- Continuous input monitoring

Logical representation:

```
Read Button State
        ↓
If Button Pressed:
    Change LED State
        ↓
Update Output
Repeat
```

This introduces the concept of event-driven control.

6. Debouncing Concept

When a mechanical button is pressed, it may generate multiple rapid signals due to physical contact vibrations. This is called **bouncing**.

To avoid multiple detections:

- A small delay is added after button press detection.

This ensures only one toggle occurs per press.

7. Significance in IoT Applications

The concepts demonstrated in this experiment form the foundation of many IoT systems, such as:

- Smart lighting systems
- Automated switches
- Security systems
- Industrial control panels

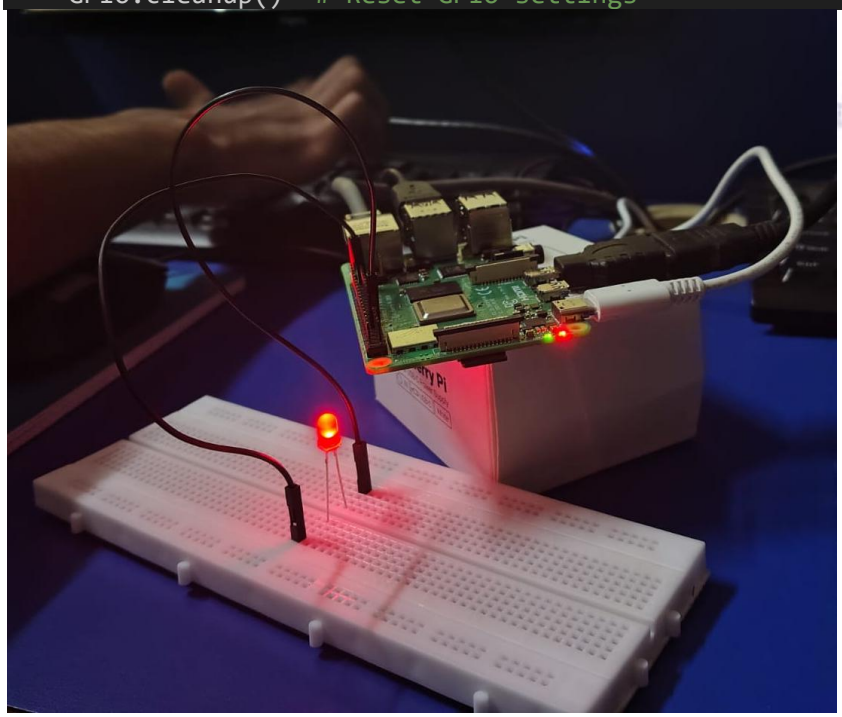
The interaction between digital input and digital output is the core principle behind sensor-actuator communication in embedded systems.

Understanding this experiment ensures a strong base for advanced interfacing and automation projects.

Code & Output:

(a) Without Using Button

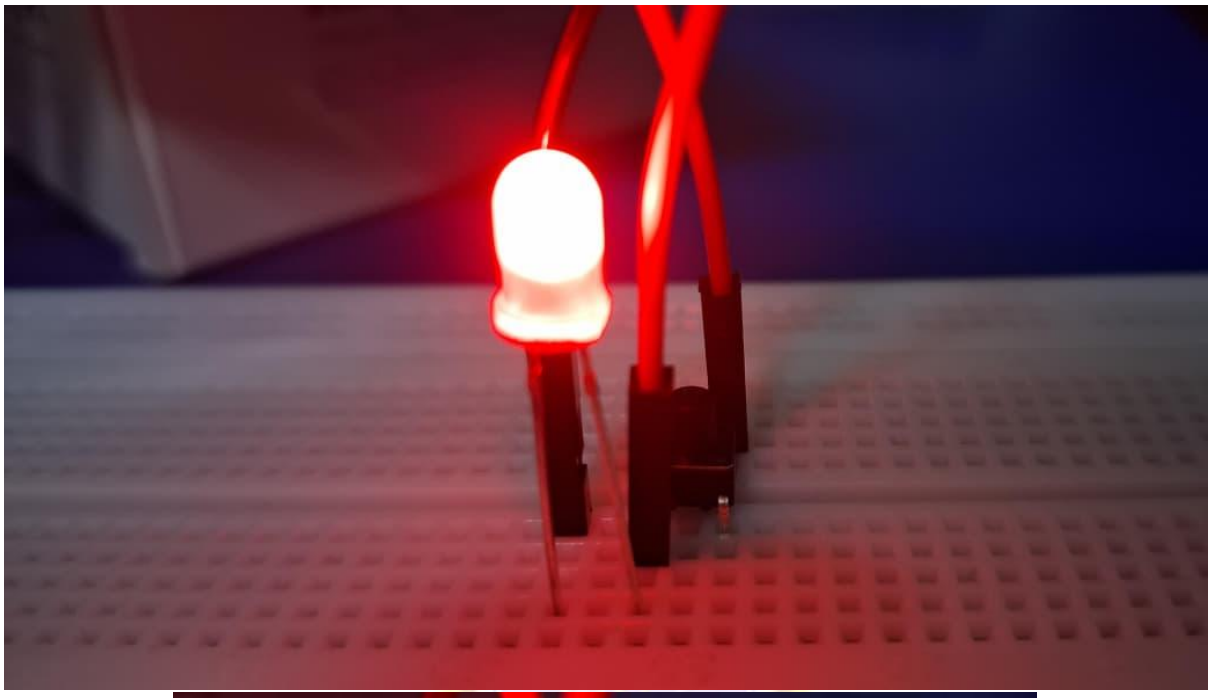
```
import RPi.GPIO as GPIO
import time

LED_PIN = 17  # GPIO17 (BCM numbering)

GPIO.setmode(GPIO.BCM)      # Use BCM pin numbering
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)  # LED ON
        time.sleep(1)                    # Wait 1 second
        GPIO.output(LED_PIN, GPIO.LOW)   # LED OFF
        time.sleep(1)                    # Wait 1 second

except KeyboardInterrupt:
    print("Program stopped")

finally:
    GPIO.cleanup()  # Reset GPIO settings
```

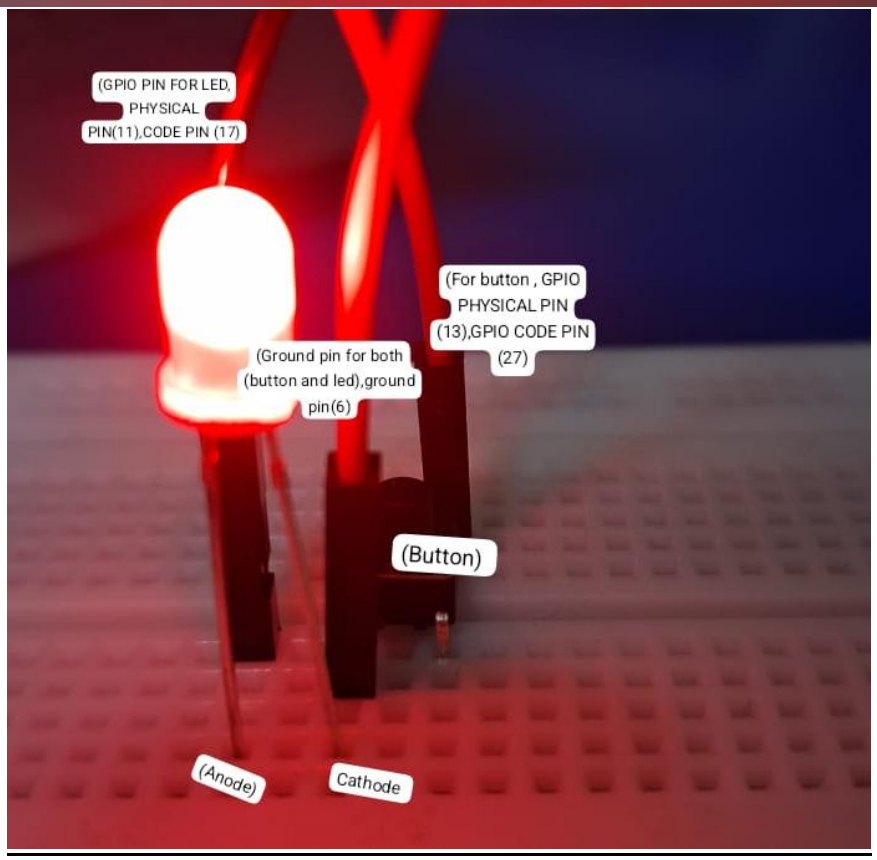


(b) With Using Button

```
import RPi.GPIO as GPIO
import time

LED = 17
BUTTON = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(BUTTON, GPIO.IN, pull_up_down=GPIO.PUD_UP)

led_state = False

try:
    while True:
        if GPIO.input(BUTTON) == GPIO.LOW:  # Button pressed
            led_state = not led_state
            GPIO.output(LED, led_state)
            time.sleep(0.3)  # debounce delay

except KeyboardInterrupt:
    GPIO.cleanup()
```

निर्मलस्नेह उत्तम सेवाधर्म




(GPIO PIN FOR LED, PHYSICAL PIN(11),CODE PIN (17)
(For button , GPIO PHYSICAL PIN (13),GPIO CODE PIN (27)
(Ground pin for both (button and led),ground pin(6)
(Button)
(Anode)
Cathode

Learning Outcomes:

1. Successfully blink an LED using Python programming on Raspberry Pi 4.
2. Interface a push button and accurately detect its pressed state.
3. Toggle an LED ON and OFF based on button input using program logic.
4. Apply GPIO configuration techniques in real-time hardware applications.
5. Build a simple interactive hardware circuit as a foundation for advanced IoT experiments.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to implement digital input and output interfacing using Raspberry Pi and Python programming. Through controlling an LED and reading push button input, students gain practical knowledge of GPIO configuration, signal processing, and logical decision-making in embedded systems. The experiment strengthens the ability to integrate hardware with software, develop real-time control applications, and design fundamental IoT-based automation systems.

Conclusion:

This experiment demonstrated the use of Raspberry Pi GPIO pins for digital input and output operations. An LED was successfully controlled using Python programming, and a push button was used to toggle the LED state. The experiment helped in understanding GPIO configuration, digital signal control, and the interaction between input devices and output devices in embedded systems.

Viva Questions:

1. What is the difference between configuring a GPIO pin as INPUT and OUTPUT?
2. Why does Raspberry Pi use 3.3V logic instead of 5V logic?
3. What is the purpose of using pull-up or pull-down configuration in input pins?
4. What is debouncing, and why is it necessary when interfacing a push button?
5. What is the difference between blinking an LED and toggling an LED using a button in terms of program logic?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—3: SPI Interface – Camera Module Integration

Aim:

a) Connect a Pi camera module (or SPI camera for Arduino)
b) Capture an image or short video
c) Store file or stream it locally

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Official Raspberry Pi 4 power adapter
- HDMI cable and monitor
- USB keyboard and mouse
- Raspberry Pi Camera Module (CSI interface)
- Camera ribbon cable

Software:

- Raspberry Pi OS
- libcamera utilities (pre-installed in latest Raspberry Pi OS)
- Python camera libraries (e.g., picamera2)

Programming Language: Python

Compiler/IDE: Thonny IDE, Terminal (for libcamera commands)

Learning Objectives:

1. Understand the working principle of SPI/CSI communication used for camera interfacing in Raspberry Pi.
2. Connect and configure the Raspberry Pi Camera Module with Raspberry Pi 4 Model B.
3. Enable and verify camera interface settings in Raspberry Pi OS.
4. Capture images and short videos using Python or terminal-based utilities.
5. Store and manage captured media files locally within the system.

Theory:

1. Introduction to Camera Integration in IoT Systems

In IoT applications, visual data plays a major role in monitoring, surveillance, automation, and object detection systems. Integrating a camera module with a controller allows the system to:

- Capture images
- Record videos
- Stream visual data
- Perform image processing

In Raspberry Pi-based systems, the camera module connects using a dedicated interface and communicates with the processor for high-speed data transfer.

This experiment focuses on:

- Connecting the Pi Camera module
- Configuring the interface
- Capturing images or short videos
- Storing the captured files locally

2. Camera Interface in Raspberry Pi

The Raspberry Pi Camera Module connects using the **CSI (Camera Serial Interface)** port. CSI is a high-speed serial interface designed specifically for camera communication.

Although SPI is a common serial protocol in embedded systems, the official Pi Camera uses the CSI interface instead of GPIO-based SPI pins.

Difference Between SPI and CSI

Feature	SPI	CSI
Full Form	Serial Peripheral Interface	Camera Serial Interface
Usage	Sensors, displays, ADCs	Camera modules
Speed	Moderate	High-speed
Data Type	General serial data	Image/video data

For image transfer, CSI is preferred due to large data size.

3. Physical Connection of Pi Camera

The camera module connects to the CSI port using a ribbon cable.

Conceptual representation:

```
+--------------------+
| Raspberry Pi 4     |
|                    |
|   CSI Port  =======|======  Camera Module
|                    |
+--------------------+
```

Connection Steps:

- Power off Raspberry Pi
- Insert ribbon cable into CSI port
- Ensure metal contacts are aligned properly
- Lock the connector

Improper insertion may result in detection failure.

4. Enabling Camera Interface

In Raspberry Pi OS, the camera interface must be enabled before usage.

Steps involve:

- Accessing system configuration
- Enabling camera support
- Rebooting the system

This ensures that the operating system recognizes the hardware module.

5. libcamera Framework

Recent versions of Raspberry Pi OS use **libcamera** instead of the older `raspistill` utility.

libcamera provides:

- Image capture command
- Video recording command
- Preview window
- Camera control options

Example functionalities:

- Capture still image
- Record video
- Adjust resolution
- Control exposure

libcamera communicates directly with the camera hardware via the CSI interface.

6. Capturing an Image

When a capture command is executed:

1. The camera sensor collects light data.
2. The data is converted into digital signals.
3. The processor processes image data.
4. The file is saved in specified format (e.g., JPG).

Logical process:

```
Initialize Camera
        ↓
Capture Frame
        ↓
Process Image
        ↓
Save File to Storage
```

7. Capturing a Short Video

For video recording:

- Multiple frames are captured per second.
- Frames are encoded into video format (e.g., .mp4).
- The file is stored in the system directory.

This process requires higher processing and storage compared to still images.

8. Storing and Accessing Captured Files

Captured files are saved in the current working directory unless specified otherwise.

Common formats:

- .jpg (image)
- .png (image)
- .mp4 (video)

Files can be accessed:

- Through File Manager
- Through terminal using file commands
- For streaming within local network

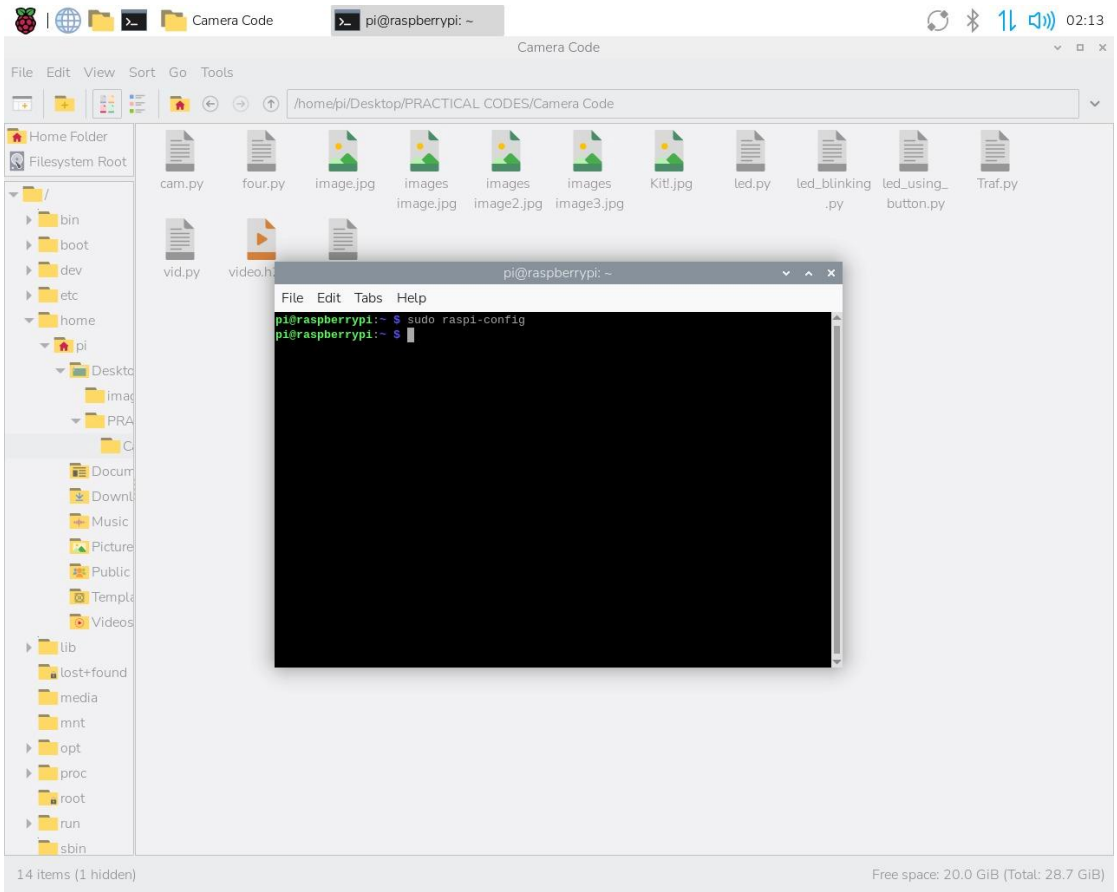
Steps, Code & Output:

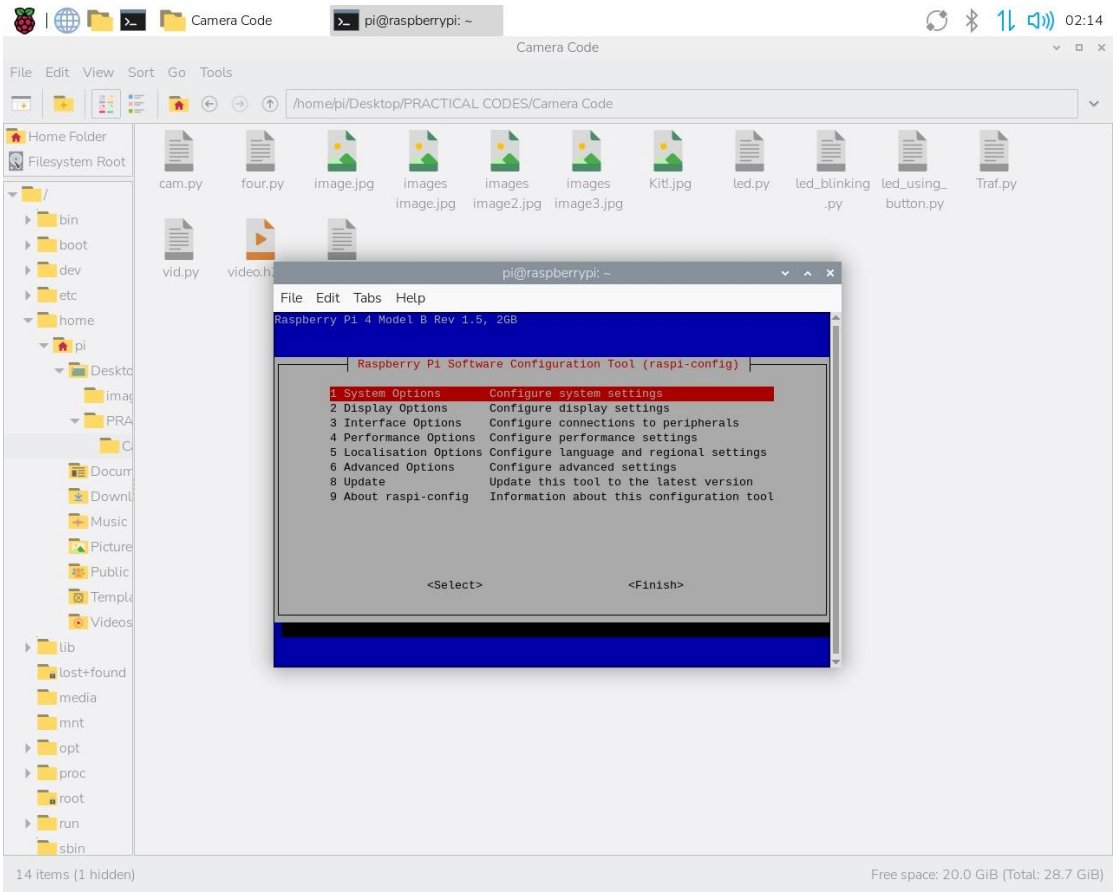
Setup The Camera (Insert The Camera In Raspberry Pi Before Starting)

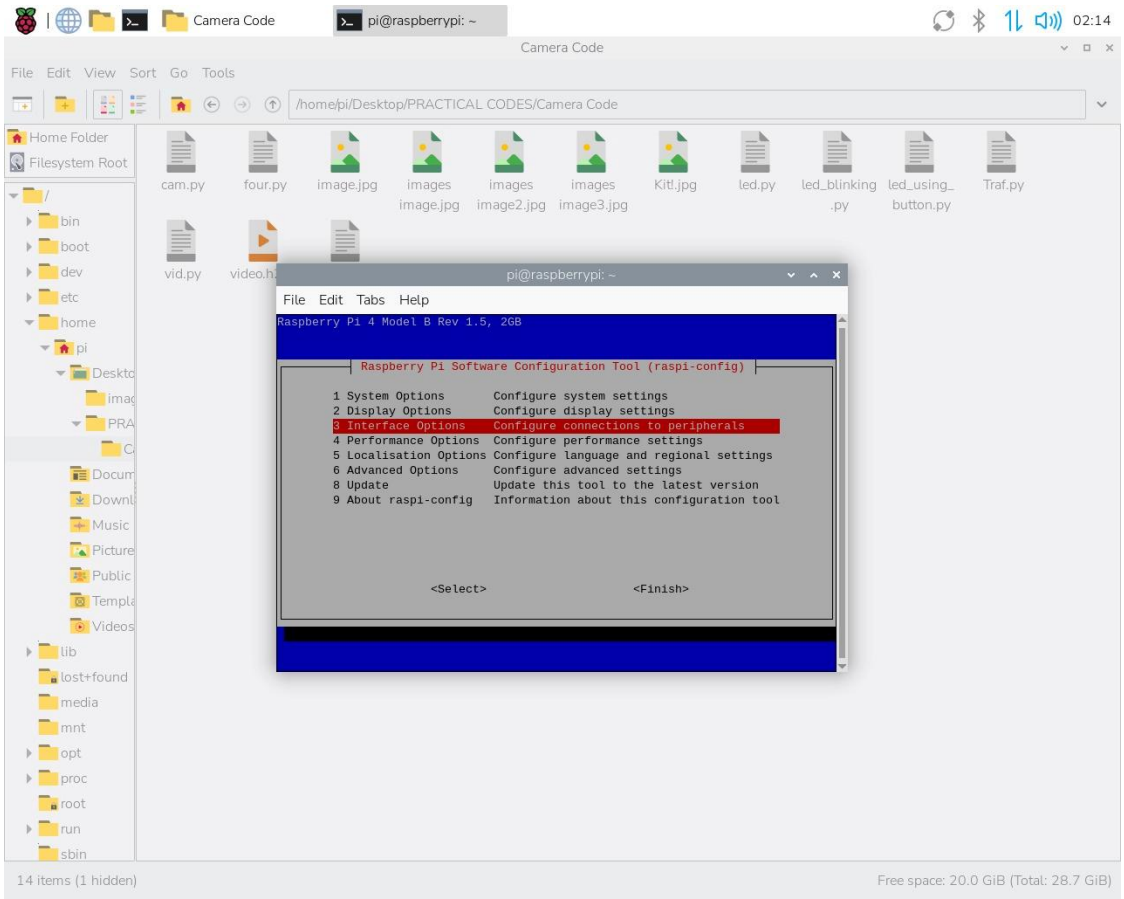
cam module commands:

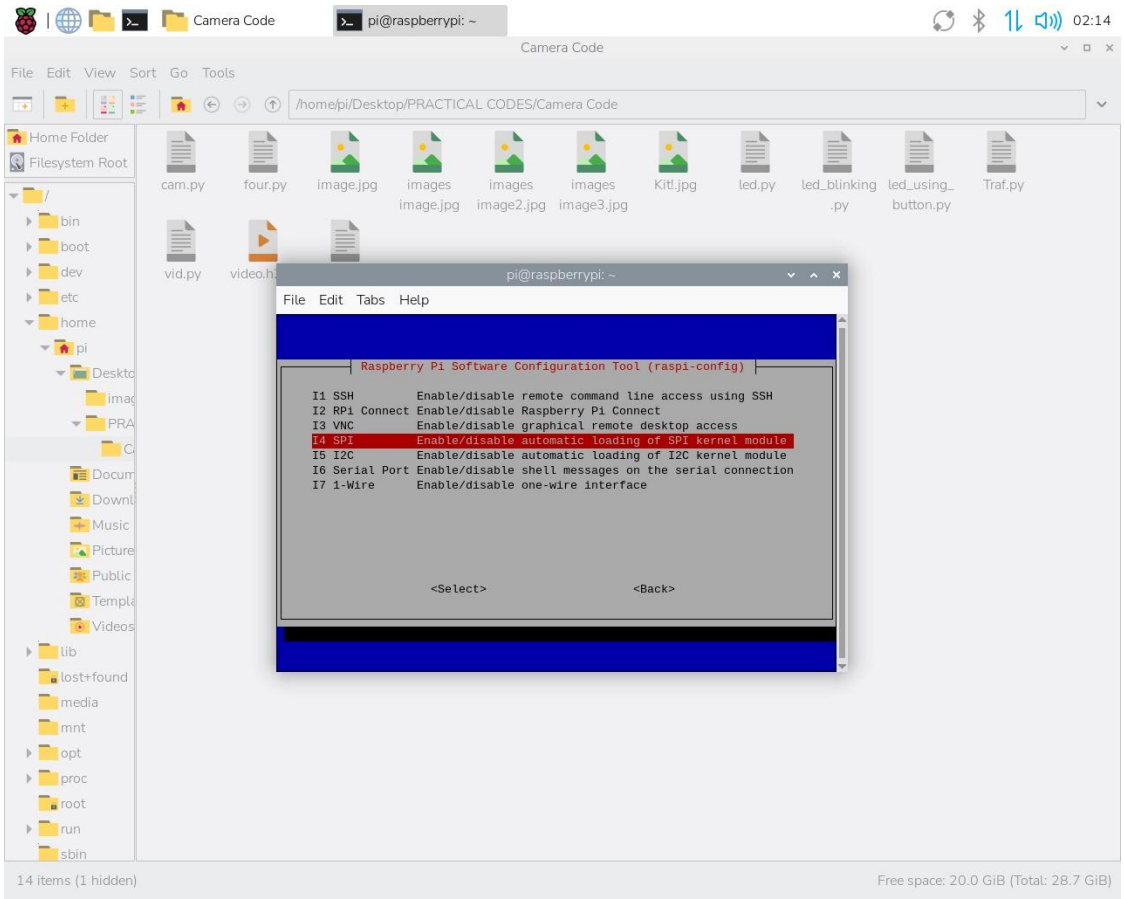
```
sudo apt update
sudo apt install libcamera-apps -y
```

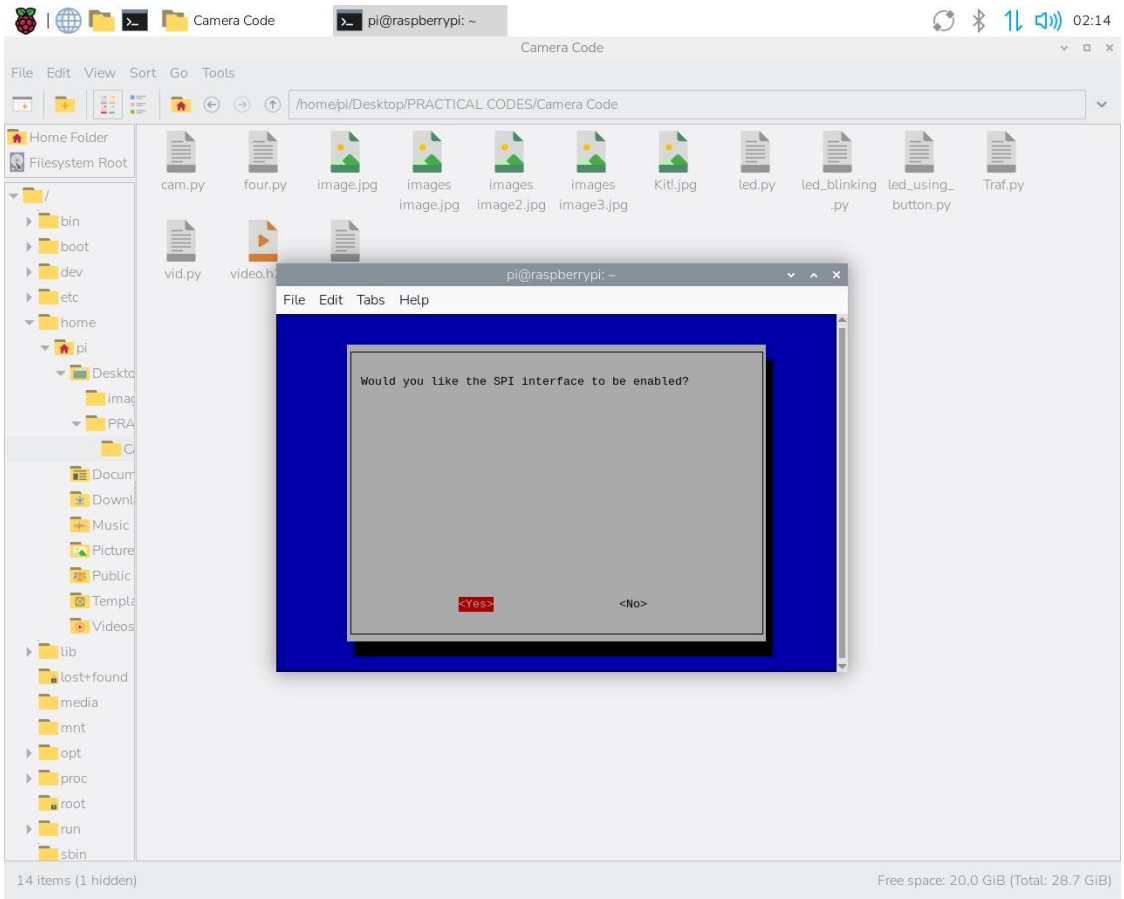
Configuring The Camera

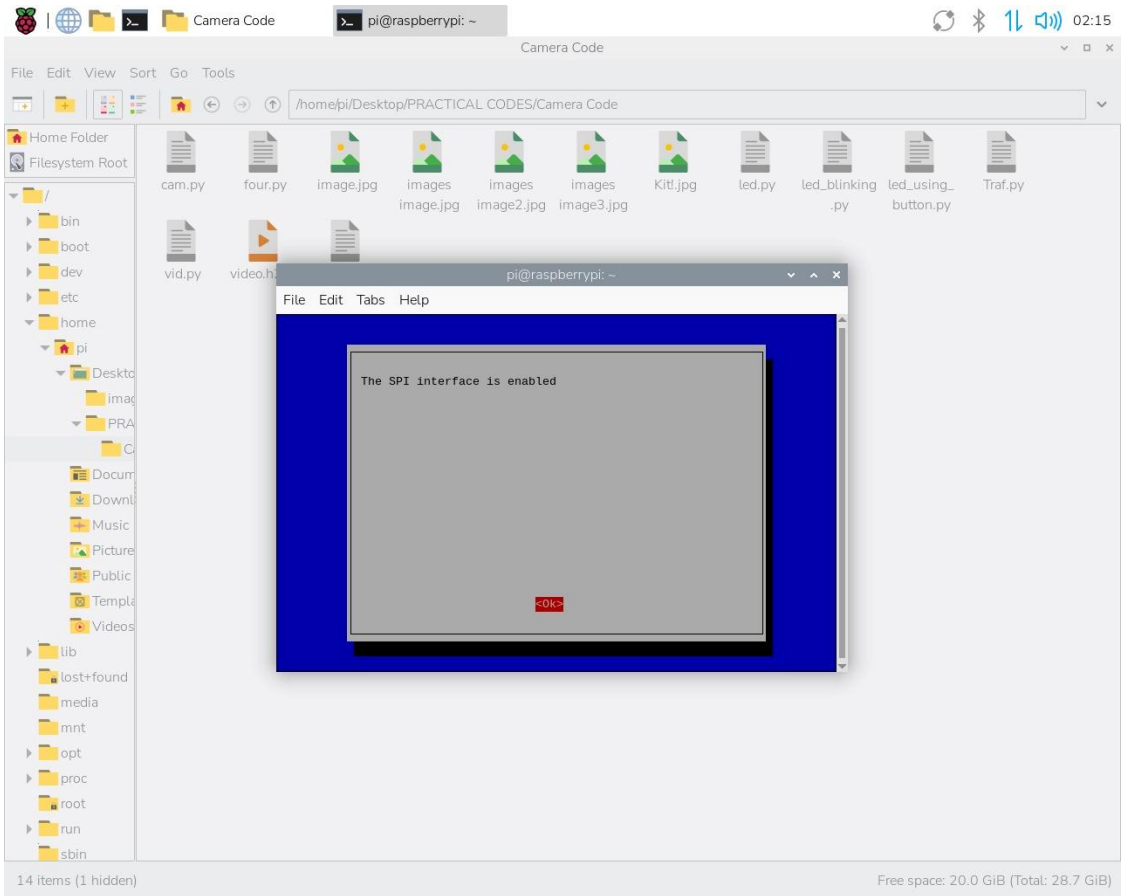



Camera Code
pi@raspberrypi: ~
02:14
Camera Code
File Edit View Sort Go Tools
/home/pi/Desktop/PRACTICAL CODES/Camera Code
Home Folder
Filesystem Root
cam.py
four.py
image.jpg
images image.jpg
images image2.jpg
images image3.jpg
Kitl.jpg
led.py
led_blinking .py
led_using_ button.py
Traf.py
vid.py
pi@raspberrypi: ~
File Edit Tabs Help
Raspberry Pi 4 Model B Rev 1.5, 2GB
Raspberry Pi Software Configuration Tool (raspi-config)
1 System Options Configure system settings
2 Display Options Configure display settings
3 Interface Options Configure connections to peripherals
4 Performance Options Configure performance settings
5 Localisation Options Configure language and regional settings
6 Advanced Options Configure advanced settings
8 Update Update this tool to the latest version
9 About raspi-config Information about this configuration tool
<Select>
<Finish>
14 items (1 hidden)
Free space: 20.0 GiB (Total: 28.7 GiB)


Camera Code
pi@raspberrypi: ~
02:14
File Edit View Sort Go Tools
/home/pi/Desktop/PRACTICAL CODES/Camera Code
Home Folder
Filesystem Root
cam.py
four.py
image.jpg
images image.jpg
images image2.jpg
images image3.jpg
Kit!.jpg
led.py
led_blinking .py
led_using_ button.py
Traf.py
vid.py
File Edit Tabs Help
Raspberry Pi 4 Model B Rev 1.5, 2GB
Raspberry Pi Software Configuration Tool (raspi-config)
1 System Options Configure system settings
2 Display Options Configure display settings
3 Interface Options Configure connections to peripherals
4 Performance Options Configure performance settings
5 Localisation Options Configure language and regional settings
6 Advanced Options Configure advanced settings
8 Update Update this tool to the latest version
9 About raspi-config Information about this configuration tool
<Select>
<Finish>
14 items (1 hidden)
Free space: 20.0 GiB (Total: 28.7 GiB)


Camera Code
pi@raspberrypi: ~
02:14
File Edit View Sort Go Tools
/home/pi/Desktop/PRACTICAL CODES/Camera Code
Home Folder
Filesystem Root
cam.py
four.py
image.jpg
images image.jpg
images image2.jpg
images image3.jpg
Kit!.jpg
led.py
led_blinking .py
led_using_ button.py
Traf.py
vid.py
File Edit Tabs Help
Raspberry Pi Software Configuration Tool (raspi-config)
I1 SSH Enable/disable remote command line access using SSH
I2 RPi Connect Enable/disable Raspberry Pi Connect
I3 VNC Enable/disable graphical remote desktop access
I4 SPI Enable/disable automatic loading of SPI kernel module
I5 I2C Enable/disable automatic loading of I2C kernel module
I6 Serial Port Enable/disable shell messages on the serial connection
I7 1-Wire Enable/disable one-wire interface
<Select>
<Back>
14 items (1 hidden)
Free space: 20.0 GiB (Total: 28.7 GiB)


Camera Code
pi@raspberrypi: ~
02:14
Camera Code
File Edit View Sort Go Tools
/home/pi/Desktop/PRACTICAL CODES/Camera Code
Home Folder
Filesystem Root
cam.py
four.py
image.jpg
images image.jpg
images image2.jpg
images image3.jpg
Kit!.jpg
led.py
led_blinking .py
led_using_ button.py
Traf.py
vid.py
pi@raspberrypi: ~
File Edit Tabs Help
Would you like the SPI interface to be enabled?
<Yes>
<No>
14 items (1 hidden)
Free space: 20.0 GiB (Total: 28.7 GiB)



Code

```
from picamera2 import Picamera2
import time

picam2 =  Picamera2()

config = picam2.create_still_configuration()
picam2.configure(config)

picam2.start()
time.sleep(2)

picam2.capture_file("/home/pi/Desktop/images/image4.jpg")
print("Image saved as image.jpg")

picam2.stop()
```

Learning Outcomes:

1. Successfully interface and detect a Pi Camera module on Raspberry Pi 4.
2. Capture images and short video clips using appropriate commands or Python programs.
3. Configure system settings required for camera operation without errors.
4. Save captured media files in the local storage directory.
5. Apply camera integration knowledge in future IoT applications such as surveillance and monitoring systems.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to interface a camera module with Raspberry Pi and perform multimedia data acquisition. Through configuring the camera interface, capturing images and videos, and storing the media locally, students gain practical knowledge of high-speed peripheral integration and system-level device management. The experiment strengthens the ability to develop IoT applications involving visual monitoring, surveillance, and data collection, while improving understanding of hardware–software interaction in embedded Linux environments.

Conclusion:

In this experiment, the Raspberry Pi camera module was successfully interfaced using the CSI interface. Images were captured using Python libraries and stored locally in the system. The experiment provided practical understanding of camera configuration, multimedia data acquisition, and file management, which are essential for IoT applications involving visual monitoring and surveillance.

Viva Questions:

1. What is the CSI interface, and why is it used for connecting the Raspberry Pi Camera Module?
2. What is the difference between SPI and CSI communication interfaces?
3. What is the purpose of the libcamera framework in Raspberry Pi OS?
4. How can images and videos captured by the camera module be stored and accessed in the system?
5. What are some real-world IoT applications that use camera modules?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—4: 8x8 LED Grid Control (Matrix LED Programming)

Aim:

a) Connect an 8×8 LED matrix module
b) Program animations or scrolling text patterns
c) Explore logical formulas for patterns

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- 8×8 LED Matrix Module (MAX7219 based)
- Jumper wires
- Breadboard

Software:

- Raspberry Pi OS
- Python LED matrix control library (such as luma.led_matrix or spidev)

Programming Language: Python

Compiler/IDE: Thonny IDE

Learning Objectives:

1. Understand the structure and functioning of an 8×8 LED matrix display.
2. Interface an LED matrix module with Raspberry Pi using suitable communication pins.
3. Program individual rows and columns to control LED patterns.
4. Generate basic animations and scrolling text using logical programming techniques.
5. Explore pattern generation concepts used in embedded display systems.

Theory:

1. Introduction to LED Matrix Displays

An **LED matrix** is an arrangement of multiple LEDs organized in rows and columns to form a grid-like structure. Instead of controlling each LED individually with separate wires, the matrix design allows multiple LEDs to be controlled using fewer input/output pins.

An **8×8 LED matrix** consists of:

- 8 rows
- 8 columns
- Total LEDs = 64

These LEDs can be controlled in combinations to display patterns, characters, symbols, or animations.

LED matrices are commonly used in:

- Digital clocks
- Information boards
- Scrolling message displays
- Game displays
- Embedded visual indicators

2. Structure of an 8×8 LED Matrix

In an 8×8 matrix, LEDs are arranged so that each LED lies at the intersection of a specific row and column.

Conceptual representation:

```
        Columns
      C1 C2 C3 C4 C5 C6 C7 C8
R1    ●  ●  ●  ●  ●  ●  ●  ●
R2    ●  ●  ●  ●  ●  ●  ●  ●
R3    ●  ●  ●  ●  ●  ●  ●  ●
R4    ●  ●  ●  ●  ●  ●  ●  ●
R5    ●  ●  ●  ●  ●  ●  ●  ●
R6    ●  ●  ●  ●  ●  ●  ●  ●
R7    ●  ●  ●  ●  ●  ●  ●  ●
R8    ●  ●  ●  ●  ●  ●  ●  ●
```

Each LED is activated by selecting a particular row and column simultaneously.

For example:

- Row 2 + Column 5 → LED at (2,5) lights up.

3. Multiplexing Concept in LED Matrix

Controlling 64 LEDs individually would require 64 GPIO pins, which is inefficient. Instead, **multiplexing** is used.

Multiplexing works by rapidly switching rows or columns so that multiple LEDs appear lit simultaneously due to persistence of vision.

Basic operation:

1. Activate Row 1
2. Send column data for Row 1
3. Activate Row 2
4. Send column data for Row 2

This process repeats very quickly.

Logical representation:

```
Activate Row
      ↓
Send Column Data
      ↓
Display LEDs
      ↓
Switch to Next Row
      ↓
Repeat Continuously
```

Because switching occurs in milliseconds, the human eye perceives a stable image.

4. MAX7219 LED Matrix Driver

Many LED matrix modules use the **MAX7219 driver chip**, which simplifies control of the matrix.

The MAX7219:

- Handles multiplexing internally
- Controls brightness
- Reduces the number of GPIO pins required
- Communicates using **SPI protocol**

Instead of controlling 64 LEDs manually, the Raspberry Pi sends pattern data to the MAX7219, which drives the LEDs.

Simplified system connection:

```
Raspberry Pi
     |
     |  SPI Communication
     |
MAX7219 Driver
     |
     |
8×8 LED Matrix
```

5. SPI Communication for LED Matrix

SPI (Serial Peripheral Interface) is a high-speed communication protocol used between the Raspberry Pi and external modules.

SPI uses four main signals:

Signal	Purpose
MOSI	Master Out Slave In
MISO	Master In Slave Out
SCLK	Clock signal
CS	Chip Select

For the LED matrix module:

- Raspberry Pi sends display data via MOSI.
- Clock signal synchronizes data transmission.
- Chip select activates the module.

This allows efficient communication with minimal wiring.

6. Displaying Patterns on LED Matrix

Patterns are created by controlling which LEDs turn ON or OFF in each row.

Example pattern (Smiley Face):

```
0 1 1 0 0 1 1 0
1 0 0 1 1 0 0 1
1 0 0 1 1 0 0 1
0 0 0 0 0 0 0 0
1 0 1 0 0 1 0 1
1 0 0 1 1 0 0 1
0 1 1 0 0 1 1 0
0 0 0 0 0 0 0 0
```

Where:

- **1 → LED ON**
- **0 → LED OFF**

The program sends these patterns to the matrix to produce images.

7. Scrolling Text on LED Matrix

Scrolling text is created by shifting patterns across the matrix.

Example:

If displaying letter **A**, its pattern moves column by column across the display.

Conceptual flow:

```
Load Character Pattern
        ↓
Display Pattern
        ↓
Shift Pattern Left
        ↓
Update Display
        ↓
Repeat
```

This technique is used in digital message boards.

8. Animations Using LED Matrix

Animations are produced by displaying a sequence of frames rapidly.

Example:

Frame 1 → Frame 2 → Frame 3 → Frame 4

Each frame slightly changes the LED pattern to create motion.

Example applications:

- Moving arrows
- Rotating shapes
- Running text

9. Importance in IoT and Embedded Systems

LED matrices are widely used for visual output in embedded devices.

Applications include:

- Smart notice boards
- Industrial status displays
- Public information systems
- IoT dashboards
- Wearable electronics

By learning LED matrix programming, students understand how visual displays can be controlled using logical programming and communication protocols.

Steps, Code & Output:

a) **Connect an 8×8 LED matrix module**

1. Identify Pins on the MAX7219 LED Matrix Module

Most MAX7219 LED matrix modules contain the following pins:

Module Pin	Function
VCC	Power supply (5V)
GND	Ground
DIN	Data Input
CS	Chip Select
CLK	Clock

2. Enable SPI Interface on Raspberry Pi

Before connecting the module, SPI must be enabled.

Open terminal and run:

```
sudo raspi-config
```

Navigate to:

```
Interface Options → SPI → Enable
```

After enabling SPI, reboot the system:

```
sudo reboot
```

3. Raspberry Pi SPI Pin Reference

Important SPI pins on Raspberry Pi:

Raspberry Pi Pin	GPIO	Function
Pin 19	GPIO10	MOSI
Pin 21	GPIO9	MISO
Pin 23	GPIO11	SCLK
Pin 24	GPIO8	CE0
Pin 2	5V	Power
Pin 6	GND	Ground

4. Hardware Connections

Connect the LED matrix module to Raspberry Pi as follows:

LED Matrix Pin	Raspberry Pi Pin
VCC	5V (Pin 2)
GND	GND (Pin 6)
DIN	GPIO10 (Pin 19)
CS	GPIO8 (Pin 24)
CLK	GPIO11 (Pin 23)

5. Connection Diagram (Conceptual)

```
Raspberry Pi                MAX7219 LED Matrix
-------------               ------------------
5V   (Pin 2)    ------------> VCC
GND  (Pin 6)    ------------> GND
GPIO10 (19)     ------------> DIN
GPIO8  (24)     ------------> CS
GPIO11 (23)     ------------> CLK
```

6. Verify SPI is Enabled

Run the following command to verify SPI devices:

```
ls /dev/spidev*
```

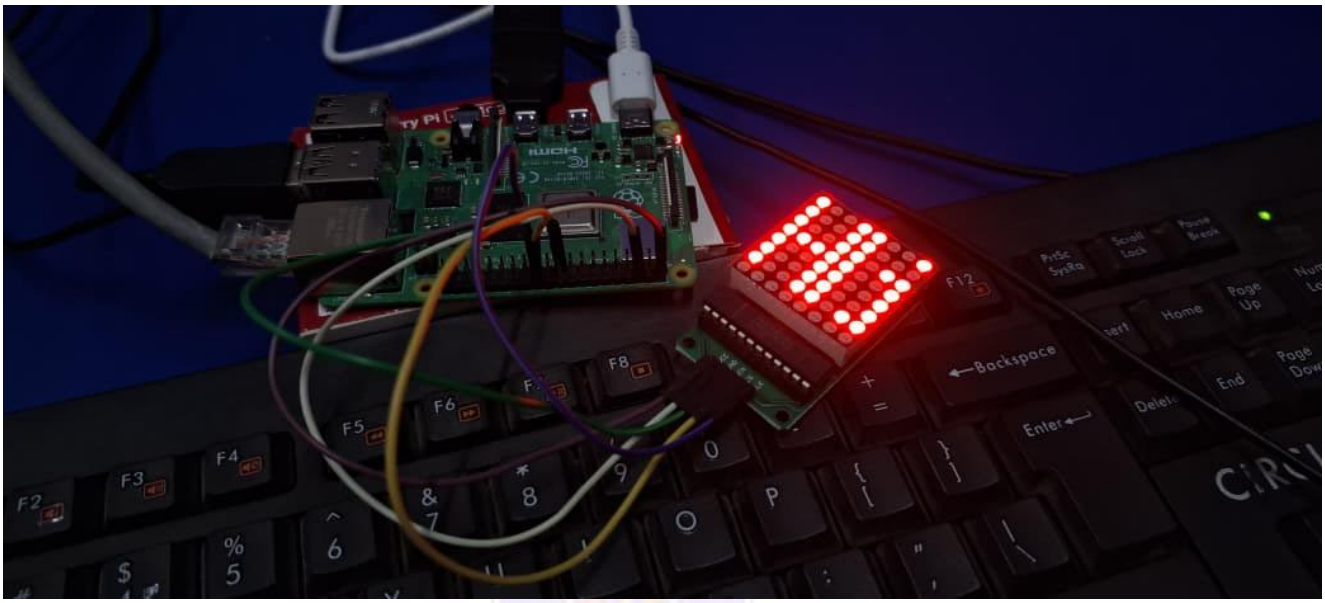
Expected output:

```
/dev/spidev0.0
/dev/spidev0.1
```

This confirms that SPI communication is ready for the LED matrix module.

b) Program animations or scrolling text patterns

```
from luma.led_matrix.device import max7219
from luma.core.interface.serial import spi, noop
from luma.core.render import canvas
from luma.core.legacy import text
from luma.core.legacy.font import proportional, CP437_FONT
import time

# SPI connection (Raspberry Pi)
serial = spi(port=0, device=0, gpio=noop())
device = max7219(serial, cascaded=1, block_orientation=0)

message = "Hi"

def scroll_text(msg, delay=1):
    msg = msg + "    "
    width = len(msg) * 8

    for offset in range(width):
        with canvas(device) as draw:
            text(draw, (-offset, 0), msg, fill="white",
                 font=proportional(CP437_FONT))
        time.sleep(delay)

while True:
    scroll_text(message)
```

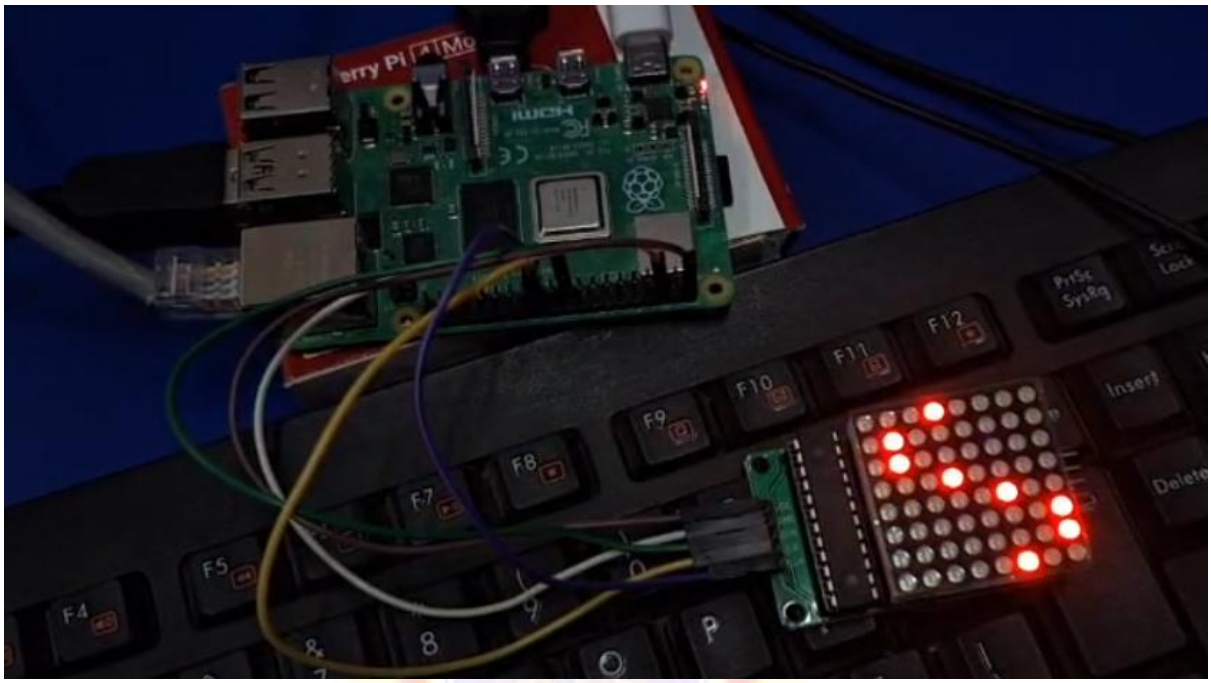


c) **Explore logical formulas for patterns**

```
from luma.led_matrix.device import max7219
from luma.core.interface.serial import spi, noop
from luma.core.render import canvas
import math
import time
import random

# SPI setup
serial = spi(port=0, device=0, gpio=noop())
device = max7219(serial, cascaded=1, block_orientation=0)

SIZE = 8

# ------------------------------
# Pattern 1: Checkerboard Logic
# pixel ON if (x + y) is even
# ------------------------------
def checkerboard():
    for _ in range(20):
        with canvas(device) as draw:
            for x in range(SIZE):
                for y in range(SIZE):
                    if (x + y) % 2 == 0:
                        draw.point((x, y), fill="white")
        time.sleep(0.4)
```

```
# ------------------------------
# Pattern 2: Diagonal Lines
# pixel ON if (x + y + t) % 4 == 0
# ------------------------------
def diagonal_lines():
    for t in range(32):
        with canvas(device) as draw:
            for x in range(SIZE):
                for y in range(SIZE):
                    if (x + y + t) % 4 == 0:
                        draw.point((x, y), fill="white")
        time.sleep(0.1)

# ------------------------------
# Pattern 3: Sine Wave Animation
# y = sin(x + t)
# ------------------------------
def sine_wave():
    for t in range(60):
        with canvas(device) as draw:
            for x in range(SIZE):
                y = int((math.sin(x * 0.8 + t * 0.3) + 1) * 3.5)
                draw.point((x, y), fill="white")
        time.sleep(0.05)

# ------------------------------
# Pattern 4: Simple Cellular Automaton
# Conway-like logic
# ------------------------------
def cellular_automaton():
    grid = [[random.randint(0,1) for _ in range(SIZE)] for _ in range(SIZE)]

    def neighbors(x, y):
        count = 0
        for dx in [-1,0,1]:
            for dy in [-1,0,1]:
                if dx == 0 and dy == 0:
                    continue
                nx = (x + dx) % SIZE
                ny = (y + dy) % SIZE
                count += grid[nx][ny]
        return count
```

```
    for _ in range(80):
        with canvas(device) as draw:
            for x in range(SIZE):
                for y in range(SIZE):
                    if grid[x][y] == 1:
                        draw.point((x, y), fill="white")

        new_grid = [[0]*SIZE for _ in range(SIZE)]
        for x in range(SIZE):
            for y in range(SIZE):
                n = neighbors(x,y)
                if grid[x][y] == 1 and n in (2,3):
                    new_grid[x][y] = 1
                elif grid[x][y] == 0 and n == 3:
                    new_grid[x][y] = 1

        grid = new_grid
        time.sleep(0.15)

# ------------------------------
# Main Loop
# ------------------------------
while True:
    checkerboard()
    diagonal_lines()
    sine_wave()
    cellular_automaton()
```



Learning Outcomes:

1. Successfully connect and operate an 8×8 LED matrix module with Raspberry Pi.
2. Control multiple LEDs simultaneously using Python programming.
3. Create simple visual patterns and animations on the LED matrix.
4. Implement scrolling text or dynamic display effects using program logic.
5. Apply LED matrix programming concepts for information display in IoT applications.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to interface and control an LED matrix display using Raspberry Pi and Python programming. Through generating visual patterns, animations, and scrolling text, students develop practical skills in digital display control, logical pattern generation, and SPI-based peripheral interfacing. The experiment enhances understanding of hardware–software interaction in embedded systems and prepares students to implement display-based feedback and visualization features in IoT applications.

Conclusion:

This experiment demonstrated how an 8×8 LED matrix module can be interfaced with Raspberry Pi using SPI communication. Various visual patterns, animations, and scrolling text were generated using Python programming and logical pattern formulas. The experiment provided insight into multiplexed display control and the use of LED matrices for information display in embedded and IoT systems.

Viva Questions:

1. What is an LED matrix, and how does an 8×8 LED matrix display work?
2. What is the role of the MAX7219 driver in controlling an LED matrix module?
3. Why is multiplexing used in LED matrix displays?
4. How are rows and columns controlled to display patterns on an LED matrix?
5. What are some practical applications of LED matrix displays in embedded and IoT systems?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—5: PWM – Stepper Motor Control

Aim:

a) Interface a stepper motor using a motor driver
b) Control direction and vary speed using PWM signals
c) Observe effect of duty cycle changes on motor movement

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- Stepper Motor (e.g., 28BYJ-48)
- Stepper Motor Driver Module (e.g., ULN2003)
- Jumper wires
- Breadboard

Software:

- Raspberry Pi OS
- RPi.GPIO Python library

Programming Language: Python

Compiler/IDE: Thonny IDE

Learning Objectives:

1. Understand the construction and working principle of a stepper motor.
2. Interface a stepper motor with Raspberry Pi using a motor driver module.
3. Generate PWM signals through GPIO pins for motor control.
4. Control the direction of the stepper motor by altering signal sequences.
5. Analyze how duty cycle variation affects the speed and movement of the motor.

Theory:

1. Introduction to Stepper Motors

A stepper motor is a type of electric motor that moves in **discrete steps** rather than continuous rotation. Each electrical pulse sent to the motor results in a fixed angular movement called a **step**.

Because of this step-by-step motion, stepper motors are widely used in systems that require **precise positioning and speed control**.

Common applications include:

- Robotics
- CNC machines
- 3D printers
- Camera positioning systems
- Industrial automation

In IoT and embedded systems, stepper motors allow precise control over mechanical movements through programmed electrical signals.

2. Basic Structure of a Stepper Motor

A stepper motor typically consists of two main parts:

- **Rotor** – The rotating part of the motor
- **Stator** – The stationary part containing electromagnets

When current flows through stator windings, magnetic fields are produced. These magnetic fields interact with the rotor and cause it to rotate step by step.

Conceptual structure:

```
+-------------------+
|       Stator      |
|   Electromagnets  |
|        ↓          |
|      Rotor        |
|   (Rotating Part) |
+-------------------+
```

Each activation of the windings pulls the rotor slightly, producing a step movement.

3. Working Principle of Stepper Motors

Stepper motors operate by energizing coils in a specific sequence.

For example:

Step	Coil A	Coil B	Coil C	Coil D
1	ON	OFF	OFF	OFF
2	OFF	ON	OFF	OFF
3	OFF	OFF	ON	OFF
4	OFF	OFF	OFF	ON

Each step energizes a different coil, causing the rotor to move incrementally.

Sequence representation:

```
Step 1 → Coil A active
Step 2 → Coil B active
Step 3 → Coil C active
Step 4 → Coil D active
```

Repeating this sequence produces continuous rotation.

4. Motor Driver Requirement

A Raspberry Pi GPIO pin cannot directly drive a stepper motor because:

- GPIO provides limited current
- Motors require higher current and voltage

Therefore, a **motor driver module** is used.

In this experiment, the **ULN2003 driver module** is commonly used.

Functions of the driver:

- Amplifies current signals
- Protects Raspberry Pi from high current
- Controls motor coils safely

Basic interfacing concept:

```
Raspberry Pi GPIO  →  ULN2003 Driver  →  Stepper Motor
        Control Signals       Power Drive
```

The driver acts as an intermediary between the controller and the motor.

5. Pulse Width Modulation (PWM)

PWM is a technique used to control the amount of power delivered to a device by rapidly switching the signal ON and OFF.

Instead of continuously varying voltage, PWM varies the **time duration of the ON signal**.

Key terms:

Term	Description
Frequency	Number of cycles per second
Duty Cycle	Percentage of ON time in one cycle

Example:

Duty Cycle	Signal Behavior
25%	Signal ON for 25% of the time
50%	Equal ON and OFF time
75%	ON most of the time

Signal representation:

```
25% Duty Cycle
|---|       |---|
    |-------|

50% Duty Cycle
|----|----|----|

75% Duty Cycle
|------|  |------|
```

6. Effect of PWM on Motor Speed

PWM controls the effective power supplied to the motor.

Relationship between duty cycle and speed:

Duty Cycle	Motor Behavior
Low duty cycle	Slow rotation
Medium duty cycle	Moderate speed
High duty cycle	Faster rotation

Increasing duty cycle increases the average voltage delivered to the motor, resulting in higher speed.

7. Controlling Direction of Stepper Motor

Stepper motors change direction depending on the sequence of signals.

Forward sequence example:

```
A → B → C → D
```

Reverse sequence example:

```
D → C → B → A
```

By reversing the order of activation, the motor rotates in the opposite direction.

This makes stepper motors highly suitable for precise motion control systems.

8. Significance of Stepper Motors in IoT Systems

Stepper motors are widely used in IoT and embedded applications that require **accurate mechanical control**.

Examples include:

- Smart camera positioning systems
- Automated doors and locks
- Robotic arms
- Smart agriculture equipment
- Industrial automation systems

Their ability to move in controlled steps makes them ideal for precision tasks.

Theoretical Steps, Code & Expected Output:

Theoretical Steps (Procedure)

1. Connect the **stepper motor to the ULN2003 driver module**.
2. Connect the **driver module input pins (IN1, IN2, IN3, IN4)** to Raspberry Pi GPIO pins.
3. Connect the **VCC and GND of the driver module** to the appropriate power supply.
4. Boot the Raspberry Pi and open **Thonny IDE**.
5. Import the required Python libraries such as `RPi.GPIO` and `time`.
6. Configure GPIO mode using **BCM numbering**.
7. Set the GPIO pins connected to the driver module as **OUTPUT**.
8. Define the step sequence required to rotate the motor.
9. Implement PWM or delay control to regulate motor speed.
10. Execute the program and observe motor rotation.
11. Reverse the sequence of signals to change the direction of rotation.
12. Modify delay values to study the effect of duty cycle on motor speed.

Python Code (Stepper Motor Control)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

# GPIO pins connected to ULN2003
motor_pins = [17, 18, 27, 22]

for pin in motor_pins:
    GPIO.setup(pin, GPIO.OUT)
    GPIO.output(pin, 0)

# Step sequence
step_sequence = [
    [1,0,0,0],
    [0,1,0,0],
    [0,0,1,0],
    [0,0,0,1]
]

try:
    while True:
        for step in step_sequence:
```

```
            for i in range(4):
                GPIO.output(motor_pins[i], step[i])
            time.sleep(0.01)  # Controls speed

except KeyboardInterrupt:
    GPIO.cleanup()
```

Expected Output

Hardware Observation

- The stepper motor begins to rotate in the forward direction.
- Each sequence activates one coil at a time, causing step-by-step movement.
- Decreasing the delay value increases the motor speed.
- Reversing the step sequence results in reverse rotation of the motor.

Learning Outcomes:

1. Successfully interface and operate a stepper motor using Raspberry Pi.
2. Control motor rotation direction through programmed signal sequences.
3. Adjust motor speed by modifying PWM duty cycle.
4. Understand the relationship between electrical control signals and mechanical motion.
5. Apply stepper motor control techniques in automation and IoT-based motion control systems.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to interface and control electromechanical devices using Raspberry Pi and Python programming. Through generating PWM signals and controlling the direction and speed of a stepper motor using a motor driver, students develop practical knowledge of motion control, signal modulation, and hardware–software integration. The experiment strengthens the ability to design embedded control systems and implement automation mechanisms in IoT-based applications involving precise mechanical movement.

Conclusion:

In this experiment, the concept of stepper motor control using a motor driver module was studied theoretically. The program demonstrated how GPIO signals can be used to control the sequence of motor coil activation to produce rotational movement. The experiment helped in understanding motor interfacing, direction control, and the relationship between signal timing and motor speed in automation systems.

Viva Questions:

1. What is a stepper motor, and how is it different from a DC motor?
2. Why is a motor driver such as ULN2003 required when interfacing a stepper motor with Raspberry Pi?
3. What is Pulse Width Modulation (PWM), and how does it help in controlling motor speed?
4. How can the direction of rotation of a stepper motor be changed in a program?
5. How does changing the duty cycle or delay affect the movement and speed of the stepper motor?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—6: Node-RED for IoT Dashboard

Aim:

a) Install and configure Node-RED on Raspberry Pi
b) Create a flow to turn LED ON/OFF via browser
c) Add visual interface for sensor data (e.g., temperature)

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- Breadboard
- LED
- DHT11 Temperature and Humidity Sensor
- Jumper wires

Software:

- Raspberry Pi OS
- Node-RED
- Node-RED Dashboard nodes
- Web browser (for accessing the dashboard)

Programming Language: JavaScript (for Node-RED function nodes), Python (for reading DHT11 sensor data)

Compiler/IDE: Node-RED Web Editor, Thonny IDE

Learning Objectives:

1. Understand the concept and architecture of Node-RED as a flow-based IoT development tool.
2. Install and configure Node-RED on Raspberry Pi for building IoT dashboards.
3. Create flows to control hardware devices such as an LED through a web interface.
4. Interface a DHT11 sensor to obtain temperature and humidity data.
5. Visualize sensor data on a browser-based dashboard using Node-RED nodes.

Theory:

1. Introduction to Node-RED in IoT

Node-RED is a **flow-based development tool** used for connecting hardware devices, APIs, and online services in IoT applications. It provides a **visual programming environment** where users can create programs by linking nodes together instead of writing large amounts of code.

Node-RED runs on **Node.js** and is commonly used with devices such as **Raspberry Pi** for rapid development of IoT systems.

Key characteristics of Node-RED include:

- Browser-based flow editor
- Drag-and-drop programming interface
- Easy integration with hardware and web services
- Real-time dashboard creation

Node-RED simplifies IoT development by allowing users to design logic flows visually.

2. Node-RED Architecture

Node-RED works based on **nodes and flows**.

- **Node** – A block that performs a specific function
- **Flow** – A collection of nodes connected together
- **Message** – Data passed between nodes

Conceptual representation:

```
Input Node → Processing Node → Output Node
```

Example:

```
Button Node → Function Node → LED Output Node
```

Data flows through connected nodes to perform specific tasks.

3. Node-RED Editor Interface

Node-RED provides a graphical editor accessible through a web browser.

Typical access address:

```
http://<RaspberryPi-IP>:1880
```

Main components of the editor:

Component	Description
Node Palette	List of available nodes
Workspace	Area where flows are created
Sidebar	Displays debug messages and configuration
Deploy Button	Runs the created flow

Users drag nodes from the palette and connect them to create an IoT application.

4. Node-RED Dashboard

Node-RED allows the creation of **web-based dashboards** for monitoring and controlling IoT devices.

The **Node-RED Dashboard** provides UI components such as:

- Buttons
- Switches
- Sliders
- Gauges
- Charts
- Text displays

These elements help create an interactive user interface for IoT systems.

Example dashboard structure:

```
Browser Dashboard
      |
      ↓
Switch → Controls LED
Gauge  → Displays Temperature
Chart  → Displays Humidity
```

The dashboard can be accessed through:

```
http://<RaspberryPi-IP>:1880/ui
```

5. LED Control Using Node-RED

In this experiment, Node-RED is used to control an LED connected to Raspberry Pi GPIO.

Working principle:

1. User presses a button on the web dashboard.
2. The command is sent to Node-RED.
3. Node-RED processes the message.
4. GPIO node turns the LED ON or OFF.

Logical flow:

```
Dashboard Button → GPIO Node → LED
```

When the button is pressed, a signal is sent to the GPIO pin controlling the LED.

6. DHT11 Temperature and Humidity Sensor

The **DHT11** is a digital sensor used for measuring:

- Temperature
- Relative Humidity

It contains:

- A humidity sensing component
- A thermistor for temperature measurement
- A digital signal processing unit

Basic pin configuration:

Pin	Function
VCC	Power supply
Data	Digital data output
GND	Ground

Conceptual connection:

```
Raspberry Pi GPIO → Data Pin (DHT11)
3.3V → VCC
GND → GND
```

The sensor sends digital data representing temperature and humidity values.

7. Displaying Sensor Data on Dashboard

Node-RED can receive sensor data and display it using dashboard widgets.

Example components used:

- **Gauge** → Shows temperature value
- **Text node** → Displays humidity reading
- **Chart node** → Shows changes over time

Data flow example:

```
DHT11 Sensor → Processing Node → Dashboard Display
```

This allows real-time monitoring through a web browser.

8. Advantages of Node-RED in IoT Systems

Node-RED is widely used in IoT development because:

- It reduces coding complexity
- Rapid prototyping is possible
- It integrates easily with sensors and cloud services
- It provides visual dashboards for monitoring

These features make Node-RED suitable for both beginners and professional IoT developers.

9. Applications of Node-RED

Node-RED is used in various IoT applications such as:

- Smart home automation systems
- Environmental monitoring systems
- Industrial IoT dashboards
- Energy monitoring systems
- Remote device control platforms

Steps, Code & Output:

(a) Install and Configure Node-RED on Raspberry Pi

Steps

1. Open the **Terminal** on Raspberry Pi.
2. Install Node-RED using the following command:

```
bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)
```

3. Enable Node-RED to start automatically at boot:

```
sudo systemctl enable nodered.service
```

4. Start Node-RED service:

```
node-red-start
```

5. Open a web browser and access the Node-RED editor:

```
http://localhost:1880
```

Output

- Node-RED starts successfully in the terminal.
- The **Node-RED editor interface** opens in the browser showing:
  - Node palette
  - Flow workspace
  - Deploy button

(b) Create a Flow to Turn LED ON/OFF via Browser

Connection

Component	Raspberry Pi
LED Anode	GPIO17
LED Cathode	GND

Steps

1. Open the **Node-RED editor** in the browser.
2. Install **node-red-dashboard** from **Manage Palette**.
3. Drag the following nodes into the workspace:
   - **Dashboard Button Node**
   - **Raspberry Pi GPIO Output Node**
4. Configure the GPIO Output node:
   - Pin → GPIO17
5. Connect nodes as:

```
Dashboard Button → GPIO Output Node → LED
```

6. Configure button payload values:
   - **1 → LED ON**
   - **0 → LED OFF**
7. Click **Deploy**.

Flow Logic

```
Browser Dashboard Button
          ↓
   Send Payload (1 or 0)
          ↓
    GPIO Output Node
          ↓
       LED ON/OFF
```

Output

Open the dashboard:

```
http://localhost:1880/ui
```

- Press **ON button** → LED turns ON
- Press **OFF button** → LED turns OFF

This confirms that the LED can be controlled remotely using a browser-based dashboard.

(c) Add Visual Interface for Sensor Data (Temperature and Humidity)

Sensor Used

DHT11 Temperature and Humidity Sensor

Connection

DHT11 Pin	Raspberry Pi
VCC	3.3V
DATA	GPIO4
GND	GND

Steps

1. Open the **Node-RED editor**:

```
http://localhost:1880
```

2. Install required nodes from **Manage Palette**:
   - `node-red-dashboard`
   - `node-red-contrib-dht-sensor`
3. Drag the following nodes into the workspace:
   - **Inject Node**
   - **DHT Sensor Node**
   - **Dashboard Gauge Node**
   - **Dashboard Text Node**
4. Configure the **DHT Sensor Node**:
   - Sensor type → **DHT11**
   - GPIO pin → **GPIO4**
5. Connect nodes as:

```
Inject Node → DHT Sensor Node → Dashboard Gauge / Text
```

6. Click **Deploy**.

Flow Logic

```
Inject Node
      ↓
DHT11 Sensor Node
      ↓
Dashboard Gauge / Text
      ↓
Display Temperature & Humidity
```

Output

Open the dashboard:

`http://localhost:1880/ui`

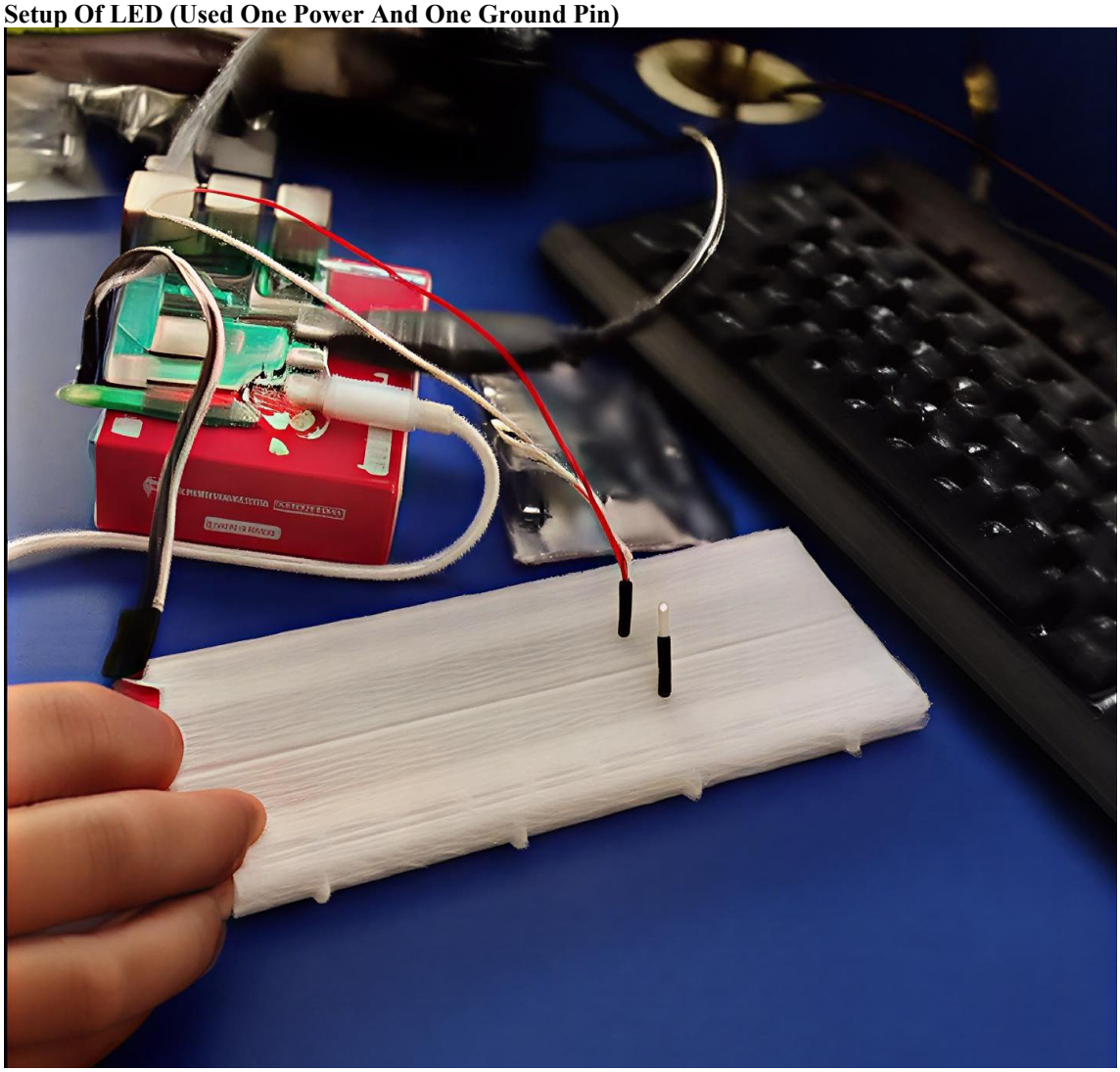
Example display:

Parameter	Value
Temperature	26 °C
Humidity	58 %

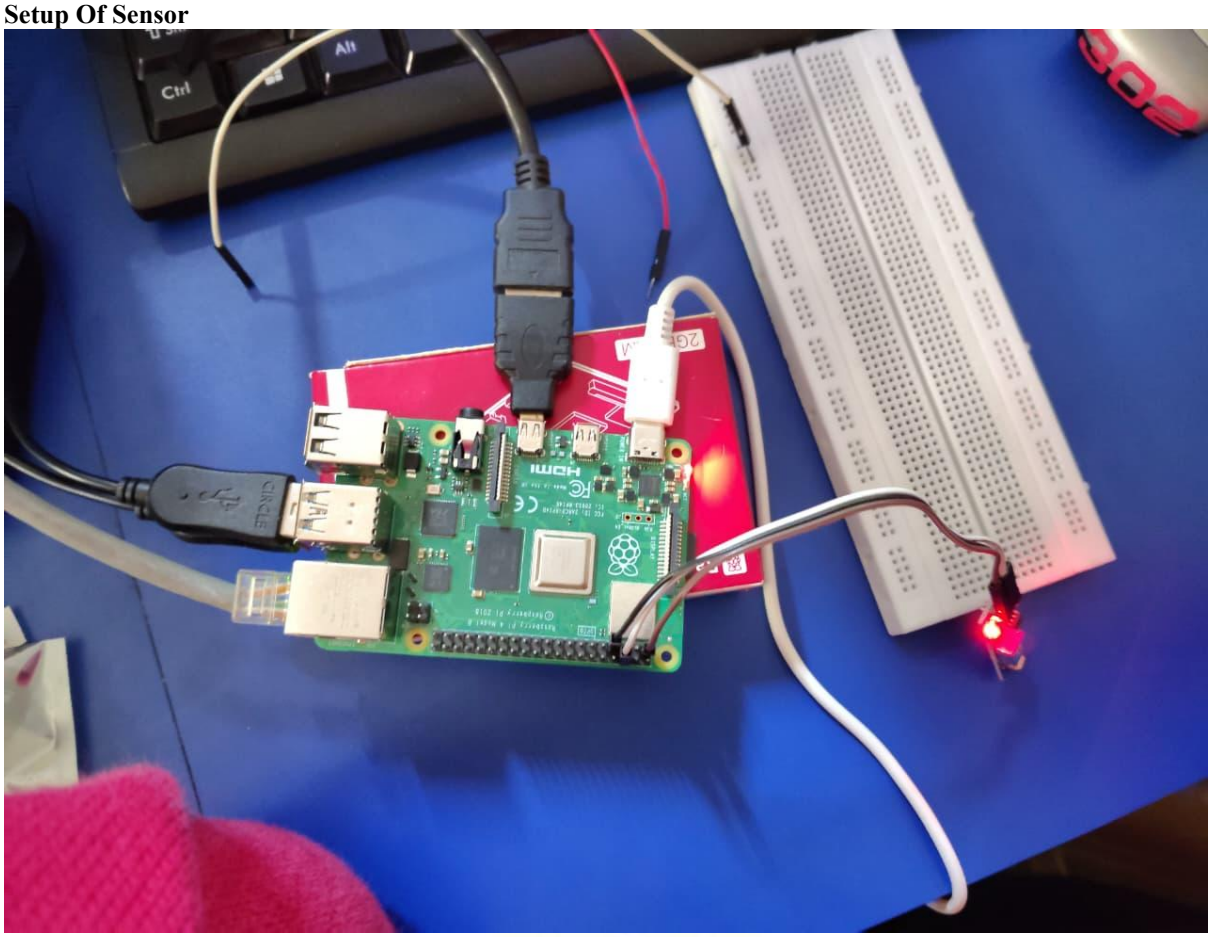
- The dashboard shows real-time **temperature and humidity values**.
- Data updates whenever the sensor reading is triggered.

IMAGE OF NODE RED SETUP OF BOTH AIM B and C

Setup Of LED (Used One Power And One Ground Pin)



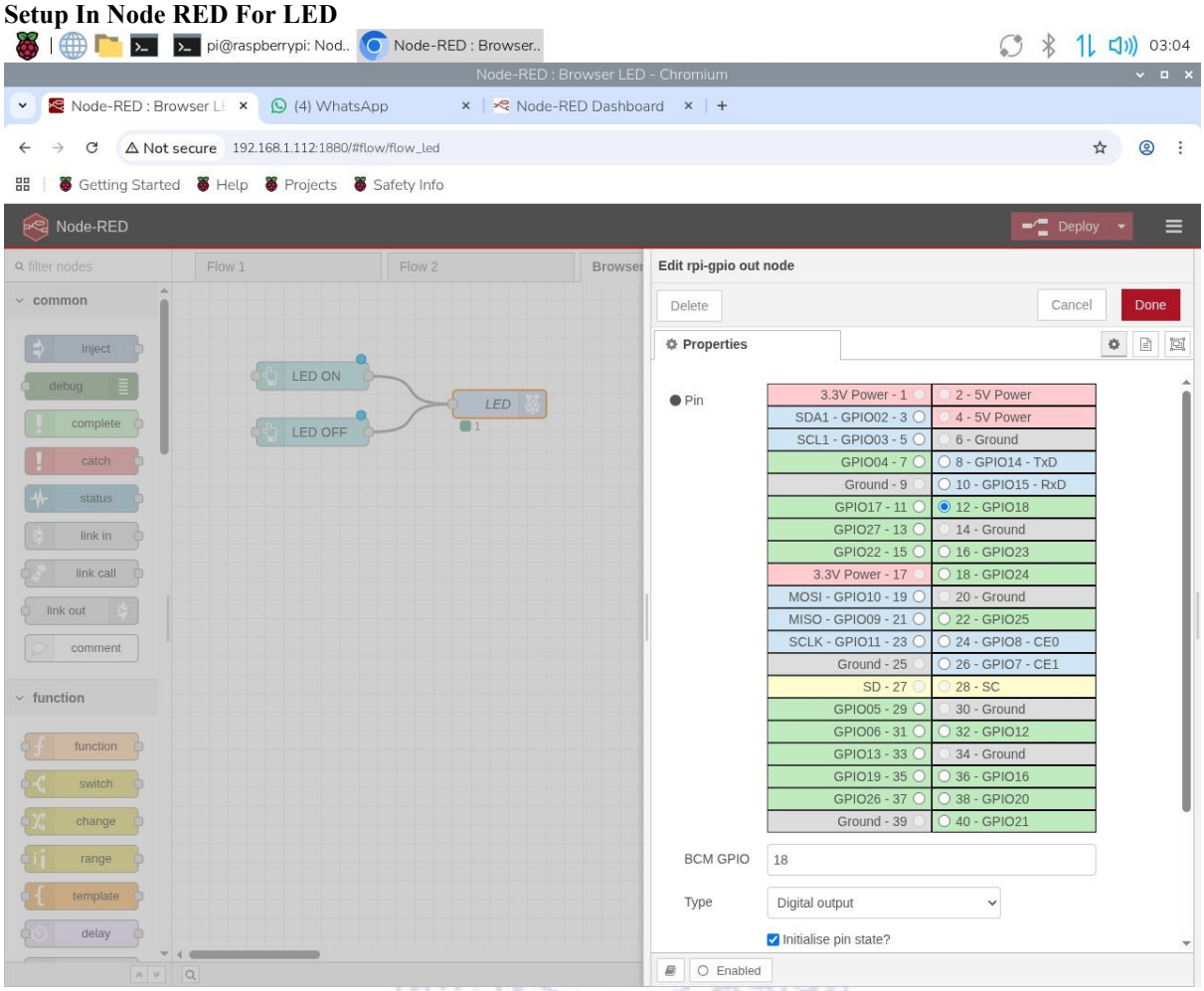
Setup Of Sensor

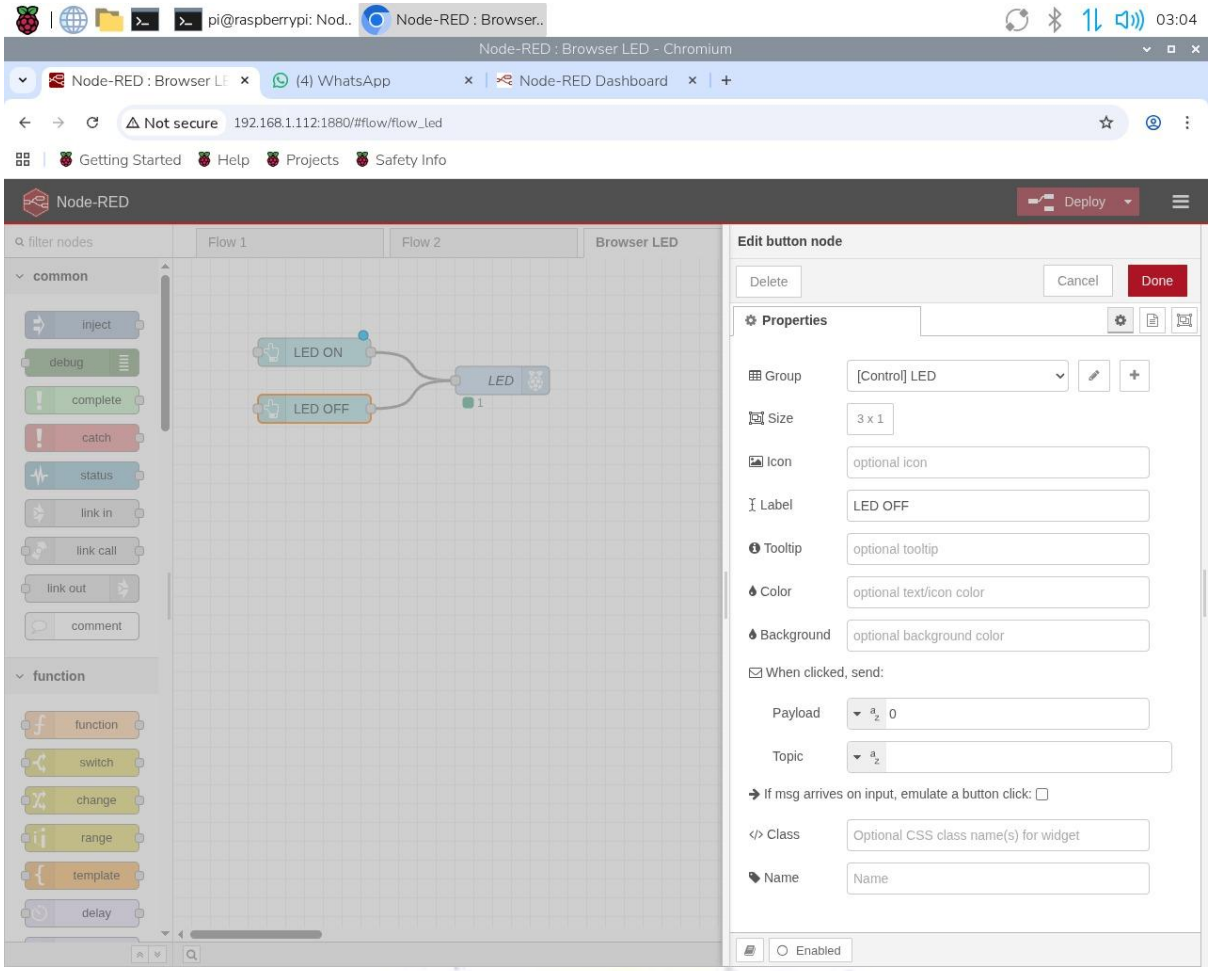


(Note That The LED Is Supposed To be Connected Together In This Connection)

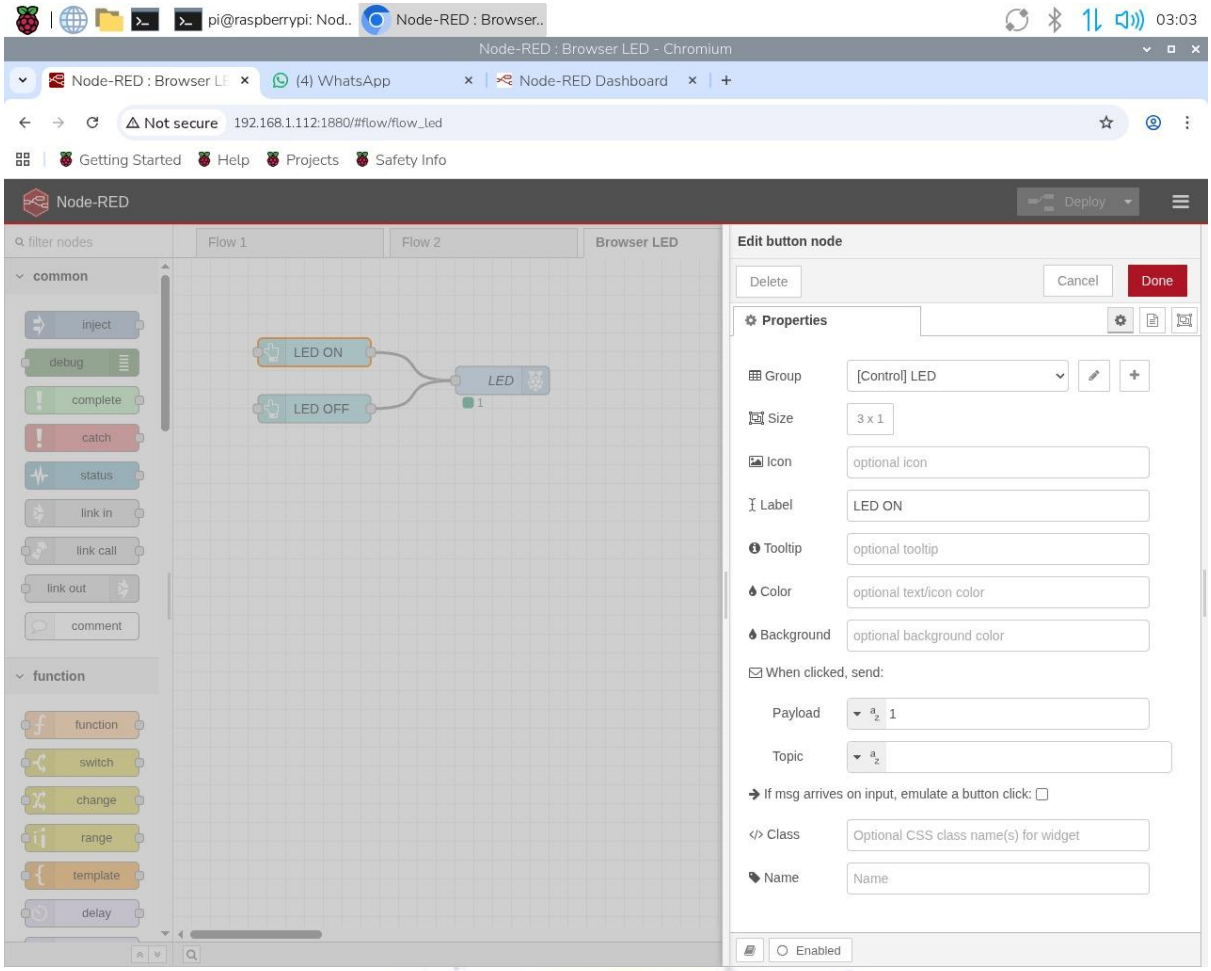
निर्मलस्नेह उत्तम सेवाधर्म

Setup In Node RED For LED



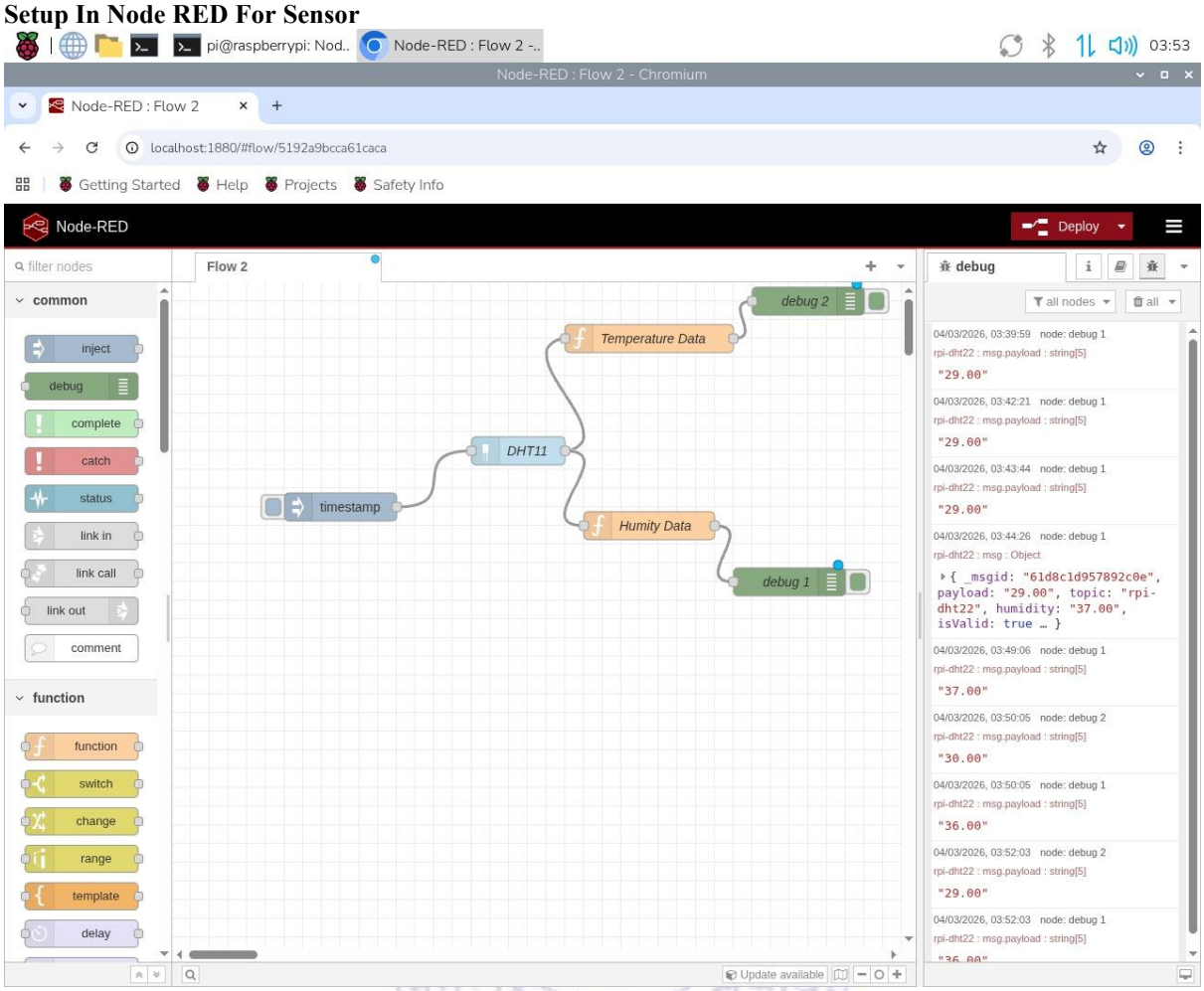


निर्मलस्नेह उत्तम सेवाधर्म

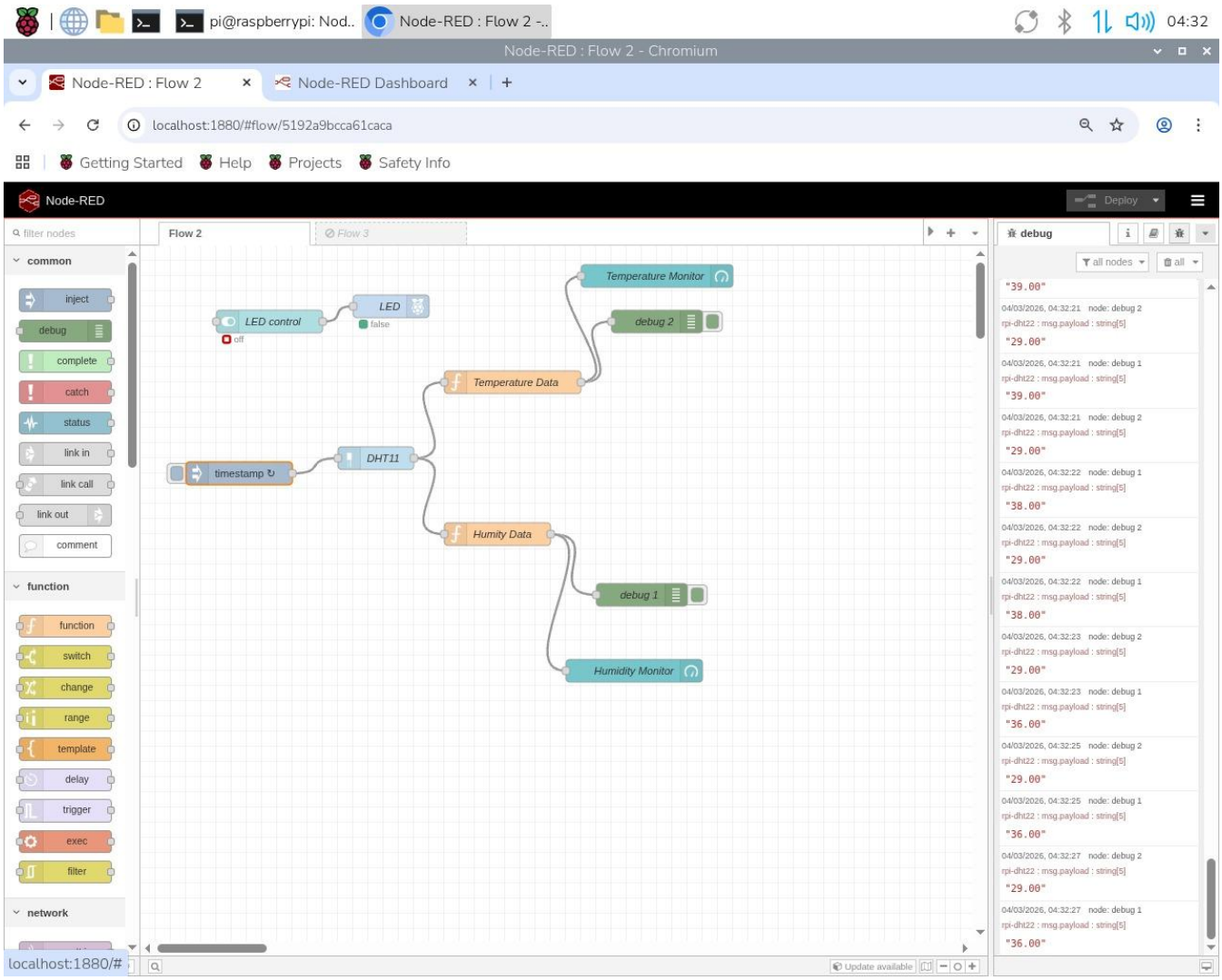


निर्मलस्नेह उत्तम सेवाधर्म

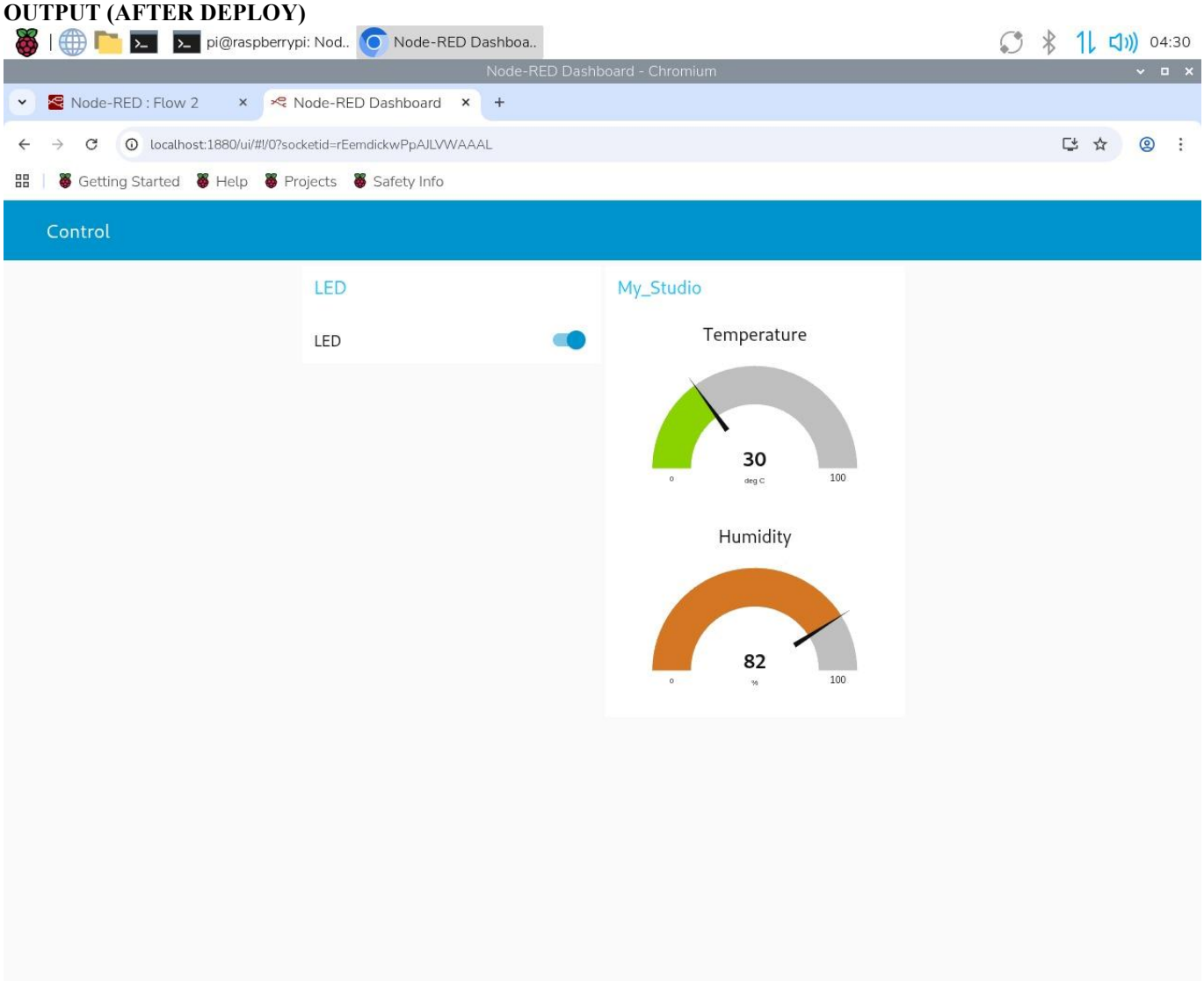
Setup In Node RED For Sensor



BOTH TOGETHER



OUTPUT (AFTER DEPLOY)



Learning Outcomes:

1. Successfully set up and run Node-RED on Raspberry Pi.
2. Design and deploy simple Node-RED flows for device control.
3. Control an LED remotely using a browser-based interface.
4. Read and display temperature and humidity values from a DHT11 sensor.
5. Develop interactive IoT dashboards for monitoring and controlling connected devices.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to develop IoT applications using Node-RED on Raspberry Pi. Through installing and configuring Node-RED, creating flows to control hardware devices such as LEDs, and visualizing temperature and humidity data from the DHT11 sensor on a web-based dashboard, students gain practical experience in flow-based programming, real-time data monitoring, and remote device control. The experiment strengthens understanding of integrating hardware components with web interfaces and supports the development of interactive IoT systems for monitoring and automation applications.

Conclusion:

This experiment demonstrated how Node-RED can be used to develop IoT dashboards for monitoring and controlling devices. An LED was controlled through a web-based interface, and temperature and humidity data from the DHT11 sensor were displayed on the dashboard. The experiment provided practical experience in flow-based programming, real-time monitoring, and integration of hardware devices with web interfaces.

Viva Questions:

1. What is Node-RED and why is it widely used in IoT development?
2. What is meant by a **flow** in Node-RED?
3. What is the purpose of the **Node-RED Dashboard**?
4. What type of data does the **DHT11 sensor** measure and how is it used in IoT applications?
5. What is the default port used to access the Node-RED editor and dashboard in a web browser?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—7: Sensor Integration – Analog & Digital Sensors

Aim:

a) Interface multiple sensors (LDR, DHT11, Gas)
b) Collect and display data on serial monitor / OLED / LCD
c) Trigger actions (e.g., fan ON if temp > threshold)

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- LDR (Light Dependent Resistor) Sensor Module
- DHT11 Temperature and Humidity Sensor
- Gas Sensor Module (e.g., MQ-2 / MQ-135)
- OLED Display / LCD Display Module
- Breadboard
- Jumper wires

Software:

- Raspberry Pi OS
- Required Python libraries for sensor interfacing (such as Adafruit_DHT, smbus, RPi.GPIO)

Programming Language: Python

Compiler/IDE: Thonny IDE

Learning Objectives:

1. Understand the operating principles of analog and digital sensors used in IoT systems.
2. Interface different types of sensors such as LDR, DHT11, and gas sensors with Raspberry Pi.
3. Acquire environmental data including light intensity, temperature, humidity, and gas levels.
4. Display sensor readings on output devices such as serial monitor, OLED, or LCD.
5. Implement simple automation logic based on sensor threshold conditions.

Theory:

1. Introduction to Sensor Integration in IoT Systems

Sensors are devices that detect changes in the physical environment and convert them into electrical signals that can be processed by a controller such as a Raspberry Pi. In Internet of Things (IoT) systems, sensors are essential for collecting real-world data and enabling automated decision-making.

In many practical IoT applications, multiple sensors are integrated together to monitor different environmental parameters such as light intensity, temperature, humidity, and gas concentration.

This experiment demonstrates the integration of multiple sensors:

- LDR (Light Dependent Resistor) for light detection
- DHT11 sensor for temperature and humidity measurement
- Gas sensor for detecting gas concentration in the environment

The collected sensor data can be displayed on devices such as a serial monitor, OLED display, or LCD display, and can also be used to trigger automated actions.

2. Types of Sensors: Analog and Digital

Sensors can be classified into two main types based on the nature of their output signal.

Type	Description	Example
Analog Sensor	Produces a continuous voltage signal that varies with environmental changes	LDR, Gas sensor
Digital Sensor	Produces discrete digital data (HIGH/LOW or formatted data signals)	DHT11

Analog Sensors

Analog sensors output a voltage value that changes according to environmental conditions.

Example:

- LDR changes resistance based on light intensity.
- Gas sensors produce voltage depending on gas concentration.

Digital Sensors

Digital sensors provide processed digital signals that can be directly read by a microcontroller or processor.

Example:

- DHT11 outputs temperature and humidity data through a digital communication protocol.

3. LDR (Light Dependent Resistor)

An LDR is a photoresistor whose resistance changes with the intensity of light falling on it.

Working Principle

- In **darkness**, LDR resistance is very high.
- In **bright light**, resistance decreases.

Conceptual representation:

```
Light Intensity ↑  →  Resistance ↓
Light Intensity ↓  →  Resistance ↑
```

Typical circuit concept:

```
3.3V
  |
  |
 [LDR]
  |
  +----> GPIO / ADC input
  |
 [Resistor]
  |
 GND
```

LDR sensors are commonly used in:

- Automatic street lighting systems
- Light monitoring systems
- Smart home lighting control

4. DHT11 Temperature and Humidity Sensor

The DHT11 is a digital sensor capable of measuring:

- Temperature
- Relative humidity

Internal Components

The DHT11 sensor contains:

- A humidity sensing element
- A thermistor for temperature measurement
- A signal processing unit

Pin configuration:

Pin	Function
VCC	Power supply
Data	Digital data output
GND	Ground

Conceptual connection:

```
Raspberry Pi GPIO ---- Data (DHT11)
3.3V ----------------- VCC
GND ------------------ GND
```

The sensor sends digital data containing temperature and humidity values.

5. Gas Sensor

Gas sensors such as **MQ-series sensors (MQ-2, MQ-135, etc.)** are used to detect gases like smoke, LPG, methane, or air pollutants.

Working Principle

Gas sensors contain a sensing material that changes its electrical resistance when exposed to certain gases.

Process:

```
Gas Concentration ↑ → Sensor Resistance Changes
                          ↓
                    Voltage Variation
                          ↓
                 Controller Reads Signal
```

Gas sensors are used in:

- Gas leakage detection systems
- Air quality monitoring
- Industrial safety systems

6. Displaying Sensor Data

Sensor readings can be displayed using different output devices.

Common display methods include:

- **Serial Monitor** – Displays data in the programming terminal.
- **OLED Display** – Small graphical display using I2C communication.
- **LCD Display** – Character-based display used to show sensor readings.

Example display concept:

```
Sensor Data → Raspberry Pi → Display Device
                      |
                      +---- Serial Monitor
                      |
                      +---- OLED / LCD Display
```

Example display output:

Parameter	Value
Temperature	27 °C
Humidity	60 %
Light Level	High
Gas Level	Normal

7. Threshold-Based Control

IoT systems often perform automated actions when sensor values exceed predefined thresholds.

Example condition:

```
If Temperature > Threshold
        ↓
Turn Fan ON
Else
Fan OFF
```

Logical representation:

```
Read Sensor Data
       ↓
Compare with Threshold
       ↓
Trigger Device (Fan / Alarm / LED)
```

Example automation scenarios:

Sensor Condition	Action
Temperature > 30°C	Fan ON
Gas detected	Alarm ON
Low light	Light ON

This concept is fundamental in building automated IoT systems.

8. Applications of Multi-Sensor Integration

Combining multiple sensors enables advanced IoT monitoring systems.

Common applications include:

- Smart home environmental monitoring
- Industrial safety monitoring
- Smart agriculture systems
- Air quality monitoring systems
- Smart building automation

These systems continuously monitor environmental conditions and automatically respond to changes.

Steps, Code & Output:

(a) Interface Multiple Sensors (LDR, DHT11, Gas)

1. LDR Sensor Interfacing (Light Detection)

Connection

LDR Module	Raspberry Pi
VCC	3.3V
GND	GND
DO	GPIO17

Theoretical Steps

1. Connect the LDR module to Raspberry Pi GPIO pins using jumper wires.
2. Open **Thonny IDE** on Raspberry Pi.
3. Import the required libraries (`RPi.GPIO` and `time`).
4. Configure GPIO mode using BCM numbering.
5. Set the LDR pin as an input pin.
6. Continuously read the digital output of the LDR module.
7. Print the light detection result on the serial monitor.

Python Code And Output

Thonny - /home/pi/Desktop/PRACTICAL CODES/Camera Code/LDR.py @ 4 : 12

formula 8x8grid prac.py | LDR.py

```python
import RPi.GPIO as GPIO
import time

LDR_PIN = 4

GPIO.setmode(GPIO.BCM)
GPIO.setup(LDR_PIN, GPIO.IN)

print("LDR Sensor Monitoring Started...")

try:
    while True:
        ldr_state = GPIO.input(LDR_PIN)

        if ldr_state == 1:
            print("Dark Detected")
        else:
            print("Bright Light Detected")

        time.sleep(1)

except KeyboardInterrupt:
    GPIO.cleanup()
    print("Program Stopped")
```

Shell

```
Dark Detected
Dark Detected
Dark Detected
Dark Detected
Dark Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Bright Light Detected
Dark Detected
Dark Detected
```

Local Python 3 • /usr/bin/python3

The output changes depending on the intensity of light falling on the LDR sensor.

2. DHT11 Temperature and Humidity Sensor

Connection

DHT11 Pin	Raspberry Pi
VCC	3.3V
DATA	GPIO4
GND	GND

Theoretical Steps

1. Connect the DHT11 sensor to Raspberry Pi GPIO pins.
2. Install the required library:

```
pip3 install Adafruit-DHT
```

3. Open **Thonny IDE**.
4. Import the DHT library.
5. Read temperature and humidity values from the sensor.
6. Display the sensor readings on the serial monitor.

Python Code

```
import Adafruit_DHT

sensor = Adafruit_DHT.DHT11
pin = 4

humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)

if humidity is not None and temperature is not None:
    print("Temperature:", temperature, "°C")
    print("Humidity:", humidity, "%")
else:
    print("Sensor reading failed")
```

Expected Output

Example terminal output:

```
Temperature: 27 °C
Humidity: 60 %
```

3. Gas Sensor (Theoretical Implementation)

Connection

Gas Sensor Pin	Raspberry Pi
VCC	5V
GND	GND
DO	GPIO22

Theoretical Steps

1. Connect the gas sensor module to the Raspberry Pi.
2. Configure the digital output pin as input.
3. Continuously monitor the gas sensor signal.
4. Display detection status on the terminal.

Python Code

```
import RPi.GPIO as GPIO
import time

GAS_PIN = 22

GPIO.setmode(GPIO.BCM)
GPIO.setup(GAS_PIN, GPIO.IN)

try:
    while True:
        if GPIO.input(GAS_PIN):
            print("Gas Detected")
        else:
            print("No Gas Detected")
        time.sleep(1)

except KeyboardInterrupt:
    GPIO.cleanup()
```

Expected Output

```
No Gas Detected
No Gas Detected
Gas Detected
```

(b) Collect and Display Data on Serial Monitor / OLED / LCD

Theoretical Steps

1. Connect sensors to Raspberry Pi GPIO pins.
2. Run Python programs to read sensor values.
3. Display the sensor readings on the **serial monitor (terminal)**.
4. Optionally connect an OLED or LCD display to show the values visually.

Example Serial Monitor Output

```
Temperature : 26 °C
Humidity    : 58 %
Light Level : Bright
Gas Level   : Normal
```

This confirms that the Raspberry Pi successfully collects and displays sensor data.

(c) Trigger Actions Based on Sensor Threshold (Theoretical)

In this experiment, automated device control is explained theoretically.

Example condition:

```
If Temperature > 30°C
        ↓
Turn Fan ON
Else
Fan OFF
```

Python Code (Conceptual Example)

```
import RPi.GPIO as GPIO
import Adafruit_DHT
import time

sensor = Adafruit_DHT.DHT11
pin = 4
FAN = 23

GPIO.setmode(GPIO.BCM)
GPIO.setup(FAN, GPIO.OUT)

while True:
    humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
```

```
    if temperature is not None:
        if temperature > 30:
            GPIO.output(FAN, True)
            print("Fan ON - High Temperature")
        else:
            GPIO.output(FAN, False)
            print("Fan OFF - Temperature Normal")

    time.sleep(2)
```

Expected Output

Example terminal output:

```
Temperature: 28
Fan OFF - Temperature Normal

Temperature: 32
Fan ON - High Temperature
```

This demonstrates how IoT systems can automatically respond to environmental conditions using sensor data.

Learning Outcomes:

1. Successfully integrate multiple sensors with Raspberry Pi for data acquisition.
2. Collect and interpret sensor data from different environmental parameters.
3. Display real-time sensor readings on visual output devices.
4. Develop simple control mechanisms based on sensor data thresholds.
5. Apply multi-sensor integration concepts in IoT-based monitoring and automation systems.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to integrate multiple environmental sensors such as LDR, DHT11, and gas sensors with a Raspberry Pi platform. Through collecting sensor data and displaying it on a serial monitor or display devices such as OLED or LCD, students gain practical experience in multi-sensor data acquisition and monitoring. The experiment also introduces the concept of automation through threshold-based control logic, strengthening the understanding of hardware–software integration and preparing students to develop intelligent IoT monitoring and control systems.

Conclusion:

In this experiment, the theoretical integration of multiple sensors such as LDR, DHT11, and gas sensors with Raspberry Pi was studied. The process of collecting environmental data and displaying it through serial output or display devices was understood. The experiment also introduced the concept of threshold-based automation, where sensor data can be used to trigger actions in IoT systems.

Viva Questions:

1. What is the difference between analog sensors and digital sensors? Give examples.
2. What parameters are measured by the DHT11 sensor?
3. What is the working principle of an LDR sensor?
4. What is the purpose of integrating multiple sensors in an IoT system?
5. How can sensor data be used to trigger automated actions in IoT applications?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—8: Web Trigger – Control GPIO from Web Server

Aim:

a) Host a simple Flask web app (Raspberry Pi) or ESP Web Page (Arduino)
b) Control a set of LEDs via buttons on a webpage
c) Ensure real-time response and feedback

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- Breadboard
- LEDs
- Jumper wires

Software:

- Raspberry Pi OS
- Flask web framework
- Web browser (for accessing the web application)

Programming Language: Python (for Flask web server and GPIO control), HTML (for creating the web interface)

Compiler/IDE: Thonny IDE, Terminal (for running the Flask server)

Learning Objectives:

1. Understand the concept of web-based control of hardware devices in IoT systems.
2. Develop a simple Flask-based web server on Raspberry Pi.
3. Interface LEDs with Raspberry Pi GPIO pins for remote control.
4. Create a web page with buttons to send control commands to the Raspberry Pi.
5. Implement real-time interaction between a web interface and physical devices.

Theory:

1. Introduction to Web-Based Control in IoT

In Internet of Things (IoT) systems, devices are often controlled remotely through web interfaces. A web server running on an embedded platform such as Raspberry Pi can host a webpage that allows users to interact with hardware devices using a browser.

Through a web interface, users can:

- Send commands to hardware devices
- Monitor device status
- Control sensors and actuators remotely

This experiment demonstrates how GPIO devices such as LEDs can be controlled through a web application hosted on Raspberry Pi.

2. Web Server Concept

A **web server** is a software application that handles requests from clients (web browsers) and returns responses such as web pages or data.

The interaction process is as follows:

```
User Browser
      ↓
HTTP Request
      ↓
Web Server (Flask)
      ↓
GPIO Control Logic
      ↓
Hardware Device (LED)
```

When a user presses a button on the webpage, the browser sends a request to the web server, which processes the request and controls the hardware accordingly.

3. Flask Web Framework

Flask is a lightweight web framework written in Python that allows developers to create web applications easily.

Flask is widely used in IoT projects because:

- It is simple and lightweight
- It integrates easily with Python-based hardware control libraries
- It allows quick creation of web APIs and dashboards

Basic components of a Flask application include:

Component	Description
Flask App	Main Python program running the web server
Routes	URLs that define actions to be executed
Templates	HTML pages displayed in the browser
Functions	Handle requests and control hardware

Example route structure:

```
/           → Load homepage
/led_on     → Turn LED ON
/led_off    → Turn LED OFF
```

4. GPIO Control Through Web Requests

GPIO pins on Raspberry Pi can be controlled through Python programs using libraries such as **RPi.GPIO**.

In this experiment:

- LEDs are connected to GPIO pins
- Flask routes trigger GPIO commands
- Web buttons send requests to these routes

Example logical flow:

```
Web Button Pressed
        ↓
Flask Route Triggered
        ↓
Python GPIO Command Executed
        ↓
LED State Changed
```

This allows real-time control of physical devices from a web browser.

5. Web Interface Design

The web interface is created using **HTML**, which provides buttons for user interaction.

Typical interface components include:

- Buttons for turning LEDs ON and OFF
- Status messages showing device state

Example conceptual interface:

```
+---------------------------+
|   Raspberry Pi Control    |
+---------------------------+
| LED 1 : [ON]   [OFF]      |
| LED 2 : [ON]   [OFF]      |
+---------------------------+
```

When a user clicks a button, the browser sends a request to the Flask server.

6. Real-Time Response and Feedback

For an IoT system to be effective, the response between the user action and the hardware reaction must be fast.

Real-time response is achieved by:

- Efficient Flask request handling
- Immediate GPIO signal changes
- Browser page updates or feedback messages

Example response cycle:

```
User Clicks Button
       ↓
HTTP Request Sent
       ↓
Flask Processes Request
       ↓
GPIO Output Changes
       ↓
LED Turns ON/OFF
```

This ensures that the user receives immediate feedback after interacting with the webpage.

7. Advantages of Web-Based GPIO Control

Using web servers for hardware control offers several advantages:

- Remote access from any browser
- Platform-independent interface
- Easy integration with IoT dashboards
- Scalability for controlling multiple devices

These features make web-based control suitable for smart home and industrial automation systems.

8. Applications of Web-Based IoT Control

Web-based device control is widely used in IoT systems such as:

- Smart home automation
- Remote lighting control
- Industrial machine monitoring
- Smart agriculture systems
- Energy management systems

These systems allow users to control and monitor devices from anywhere through a web interface.

Code & Output:

(a) Host a Simple Flask Web Application (Theoretical)

Steps

1. Install Flask on Raspberry Pi using the terminal.

```
pip3 install flask
```

2. Create a Python file named **app.py**.
3. Write a Flask application that hosts a simple webpage.
4. Run the Flask server using the terminal.

```
python3 app.py
```

5. Open a web browser and access the web application.

```
http://<RaspberryPi-IP>:5000
```

Code (Flask Web Server)

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to Raspberry Pi Web Server"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Expected Output

When the browser accesses the server:

```
Welcome to Raspberry Pi Web Server
```

This confirms that the Flask web server is running successfully.

(b) Control LEDs via Buttons on a Webpage (Theoretical)

Connection

Component	Raspberry Pi
LED 1	GPIO17
LED 2	GPIO18
LED Cathode	GND

Steps

1. Connect LEDs to Raspberry Pi GPIO pins.
2. Create a Flask application that includes LED control routes.
3. Design a webpage with ON and OFF buttons.
4. When a button is pressed, the Flask route changes the GPIO output state.

Code

```
from flask import Flask, render_template
import RPi.GPIO as GPIO

GPIO.setmode(GPIO.BCM)

LED1 = 17
LED2 = 18

GPIO.setup(LED1, GPIO.OUT)
GPIO.setup(LED2, GPIO.OUT)

app = Flask(__name__)

@app.route('/')
def index():
    return '''
    <h2>LED Control</h2>
    <a href="/led1_on"><button>LED1 ON</button></a>
    <a href="/led1_off"><button>LED1 OFF</button></a><br><br>
    <a href="/led2_on"><button>LED2 ON</button></a>
    <a href="/led2_off"><button>LED2 OFF</button></a>
    '''

@app.route('/led1_on')
def led1_on():
    GPIO.output(LED1, True)
    return "LED1 Turned ON"

@app.route('/led1_off')
def led1_off():
    GPIO.output(LED1, False)
    return "LED1 Turned OFF"

@app.route('/led2_on')
def led2_on():
    GPIO.output(LED2, True)
    return "LED2 Turned ON"

@app.route('/led2_off')
def led2_off():
    GPIO.output(LED2, False)
```

```
    return "LED2 Turned OFF"

if __name__ == '__main__':
    app.run(host='0.0.0.0')
```

Expected Output

Webpage Interface Example:

```
LED Control

[LED1 ON]    [LED1 OFF]

[LED2 ON]    [LED2 OFF]
```

Pressing buttons changes the LED states.

Example responses:

```
LED1 Turned ON
LED1 Turned OFF
LED2 Turned ON
LED2 Turned OFF
```

(c) Ensure Real-Time Response and Feedback (Theoretical)

Working Logic

The system provides immediate feedback when the user interacts with the webpage.

Logical process:

```
User Clicks Button
        ↓
HTTP Request Sent
        ↓
Flask Server Receives Request
        ↓
GPIO Output Updated
        ↓
LED State Changes
        ↓
Feedback Message Displayed
```

Expected Observation

Button Pressed	LED Status
LED1 ON	LED1 glows
LED1 OFF	LED1 turns off
LED2 ON	LED2 glows
LED2 OFF	LED2 turns off

This demonstrates real-time interaction between the web server and Raspberry Pi GPIO hardware.

Learning Outcomes:

1. Successfully host a Flask web application on Raspberry Pi.
2. Control LEDs connected to GPIO pins using commands from a web browser.
3. Develop simple web interfaces for IoT device control.
4. Establish communication between a web server and hardware components.
5. Apply web-based control techniques for building remote IoT automation systems.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to develop web-based control systems for IoT devices using Raspberry Pi and the Flask framework. By hosting a simple web server and controlling LEDs through a browser interface, students gain practical experience in integrating web technologies with embedded hardware systems. The experiment strengthens understanding of remote device control, real-time interaction between web applications and GPIO hardware, and the development of IoT automation systems accessible through network-based interfaces.

Conclusion:

This experiment demonstrated how a Flask-based web server can be used to control GPIO devices remotely through a web browser. LEDs connected to Raspberry Pi were controlled using buttons on a web page, providing real-time interaction between hardware and web applications. The experiment highlighted the importance of web technologies in enabling remote device control in IoT systems.

Viva Questions:

1. What is Flask and why is it commonly used in IoT applications with Raspberry Pi?
2. What is the role of a web server in controlling IoT devices?
3. What is the default port used by Flask when running a web application?
4. How does a web button trigger GPIO actions in a Flask-based application?
5. What are the advantages of controlling hardware devices through a web interface?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—9: IoT Protocol – Send Sensor Data Online

Aim:

a) Use HTTP or MQTT to push sensor values to a cloud server (e.g., Thingspeak)
b) Implement publishing logic with timestamps
c) Use Arduino/NodeMCU or Raspberry Pi as the publisher

Tools Used:

Hardware:

- Raspberry Pi 4 Model B / NodeMCU (ESP8266) / Arduino Board
- MicroSD card with Raspberry Pi OS installed (if Raspberry Pi is used)
- USB cable for programming the board
- Temperature / Environmental sensor (e.g., DHT11)
- Breadboard
- Jumper wires

Software:

- Raspberry Pi OS / Arduino IDE environment
- ThingSpeak Cloud Platform
- Required libraries for HTTP/MQTT communication

Programming Language: Python (for Raspberry Pi implementation), C/C++ (for Arduino / NodeMCU implementation)

Compiler/IDE: Thonny IDE (for Raspberry Pi Python programs), Arduino IDE (for Arduino / NodeMCU programming)

Learning Objectives:

1. Understand the importance of IoT communication protocols such as HTTP and MQTT.
2. Configure a cloud platform (e.g., ThingSpeak) for storing IoT sensor data.
3. Send sensor readings from a hardware device to an online server using HTTP or MQTT requests.
4. Implement data publishing with timestamps for proper monitoring and analysis.
5. Observe and analyze real-time sensor data through a cloud-based dashboard.

Theory:

1. Introduction to IoT Communication Protocols

In Internet of Things (IoT) systems, devices collect data from sensors and send it to remote servers or cloud platforms for monitoring, storage, and analysis. This communication is achieved using standardized **IoT communication protocols**.

Protocols define how data is transmitted between devices and servers over a network.

Commonly used IoT protocols include:

- **HTTP (HyperText Transfer Protocol)**
- **MQTT (Message Queuing Telemetry Transport)**

These protocols enable devices such as Raspberry Pi, Arduino, or NodeMCU to send sensor data to cloud platforms like **ThingSpeak**.

2. HTTP Protocol in IoT

HTTP is one of the most widely used communication protocols on the internet. It is a request-response protocol where a client sends a request to a server, and the server responds with the requested data.

In IoT applications:

- The IoT device acts as the **client**.
- The cloud platform acts as the **server**.

Example process:

```
IoT Device (Client)
        ↓
HTTP Request
        ↓
Cloud Server (ThingSpeak)
        ↓
Store Sensor Data
```

HTTP is simple to implement and works well for applications where devices periodically upload sensor data to a cloud platform.

3. MQTT Protocol in IoT

MQTT is a lightweight messaging protocol designed specifically for IoT systems with limited bandwidth and resources.

MQTT follows a **publish–subscribe model**.

Components of MQTT communication:

Component	Description
Publisher	Device that sends data
Broker	Server that manages messages
Subscriber	Device or application that receives data

Conceptual representation:

```
Publisher (Sensor Node)
        ↓
     MQTT Broker
        ↓
Subscriber (Cloud Dashboard / Application)
```

MQTT is efficient because devices do not need to continuously communicate with each other directly.

4. ThingSpeak Cloud Platform

ThingSpeak is an IoT cloud platform that allows users to:

- Collect sensor data
- Store data online
- Visualize data using graphs
- Perform data analysis

Each project in ThingSpeak is called a **channel**, and each channel contains fields where sensor values are stored.

Typical structure:

Field	Example Data
Field 1	Temperature
Field 2	Humidity
Field 3	Light Level

Sensor data is uploaded to ThingSpeak using an **API key**.

Example HTTP request format:

```
https://api.thingspeak.com/update?api_key=YOUR_API_KEY&field1=value
```

5. Data Publishing with Timestamps

When sensor data is uploaded to a cloud platform, it is stored along with the **timestamp** indicating when the data was received.

Example dataset:

Time	Temperature	Humidity
10:01	26°C	55%
10:02	27°C	56%
10:03	27°C	57%

Timestamps allow:

- Tracking of sensor trends over time
- Historical data analysis
- Real-time monitoring

Most cloud platforms automatically generate timestamps when data is received.

6. IoT Device as Publisher

In this experiment, devices such as **Raspberry Pi, Arduino, or NodeMCU** act as publishers.

The device performs the following steps:

```
Read Sensor Data
        ↓
Format Data
        ↓
Send Data using HTTP or MQTT
        ↓
Cloud Server Stores Data
        ↓
Dashboard Displays Data
```

This process enables remote monitoring of sensor readings.

7. Data Visualization on Cloud Dashboard

Cloud platforms like ThingSpeak provide graphical dashboards that display sensor data in the form of charts and graphs.

Example dashboard representation:

```
Temperature Graph
30°C |         *
28°C |       *
26°C |     *
24°C |   *
     +----------------
       Time →
```

The dashboard updates automatically whenever new sensor data is uploaded.

8. Advantages of Cloud-Based IoT Data Logging

Using cloud platforms for IoT data transmission provides several benefits:

- Remote monitoring from anywhere
- Real-time data visualization
- Secure data storage
- Easy integration with analytics tools

These features make cloud-based IoT systems highly scalable and useful for real-world applications.

9. Applications of IoT Data Publishing

Sending sensor data to cloud platforms is widely used in:

- Smart agriculture monitoring
- Weather monitoring systems
- Industrial equipment monitoring
- Smart home automation
- Environmental pollution tracking

These systems allow users to monitor conditions remotely and make informed decisions based on real-time data.

Theoretical Steps, Code & Expected Output:

(a) Use HTTP or MQTT to Push Sensor Values to a Cloud Server (ThingSpeak) – Theoretical

Steps

1. Create an account on the **ThingSpeak IoT Cloud Platform**.
2. Create a **new channel** to store sensor data.
3. Configure fields for sensor values (e.g., Temperature, Humidity).
4. Note the **Write API Key** generated for the channel.
5. Connect the IoT device (Raspberry Pi / NodeMCU / Arduino) to the internet.
6. Write a program that sends sensor values to ThingSpeak using **HTTP requests or MQTT publishing**.

Example HTTP Request Format

```
https://api.thingspeak.com/update?api_key=YOUR_API_KEY&field1=value
```

This request sends sensor data to the ThingSpeak server.

Python Code Example (Raspberry Pi – HTTP Method)

```python
import requests
import time

api_key = "YOUR_API_KEY"
url = "https://api.thingspeak.com/update"

while True:
    temperature = 26   # Example sensor value

    payload = {'api_key': api_key, 'field1': temperature}
    response = requests.get(url, params=payload)

    print("Data Sent:", temperature)

    time.sleep(15)
```

Expected Output

Terminal Output:

```
Data Sent: 26
Data Sent: 27
Data Sent: 28
```

ThingSpeak dashboard displays sensor data graphically.

Example channel graph:

Time	Temperature
10:01	26 °C
10:02	27 °C
10:03	28 °C

(b) Implement Publishing Logic with Timestamps – Theoretical

Steps

1. Collect sensor data periodically from the device.
2. Send the data to the cloud server at regular intervals.
3. Each uploaded data entry is stored along with a timestamp.
4. The timestamp indicates when the data was published.

Code Example

```
import requests
import time
from datetime import datetime

api_key = "YOUR_API_KEY"
url = "https://api.thingspeak.com/update"

while True:
    temperature = 27
    timestamp = datetime.now()

    payload = {'api_key': api_key, 'field1': temperature}
    requests.get(url, params=payload)

    print("Temperature:", temperature, "Time:", timestamp)

    time.sleep(15)
```

Expected Output

Terminal output example:

```
Temperature: 27 Time: 2024-04-10 10:01:10
Temperature: 28 Time: 2024-04-10 10:01:25
Temperature: 27 Time: 2024-04-10 10:01:40
```

ThingSpeak automatically logs data with timestamps.

Example dataset:

Timestamp	Temperature
10:01:10	27 °C
10:01:25	28 °C
10:01:40	27 °C

(c) Use Arduino / NodeMCU / Raspberry Pi as Publisher – Theoretical

In this experiment, the IoT device acts as a **publisher**, sending sensor data to the cloud server.

Publishing process:

```
Sensor → IoT Device → Internet → Cloud Server → Dashboard
```

Example architecture:

```
Sensor
   ↓
IoT Device (Publisher)
   ↓
HTTP / MQTT Protocol
   ↓
ThingSpeak Server
   ↓
Data Visualization Dashboard
```

Example MQTT Publishing Logic (Conceptual)

```
Sensor Data
      ↓
MQTT Publish Message
      ↓
MQTT Broker
      ↓
Cloud Dashboard
```

Expected Output

Cloud dashboard displays sensor values in graphical format.

Example graph representation:

```
Temperature (°C)

30 |        *
28 |      *
26 |    *
24 |  *
   +-------------------
       Time →
```

The dashboard updates automatically whenever new sensor data is published.

Learning Outcomes:

1. Successfully establish communication between an IoT device and a cloud server.
2. Upload sensor data to an online platform such as ThingSpeak.
3. Monitor real-time data and graphical visualization through the cloud dashboard.
4. Implement basic IoT data publishing mechanisms using standard communication protocols.
5. Apply cloud connectivity concepts for remote monitoring and IoT data analytics applications.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to implement device-to-cloud communication in IoT systems using protocols such as HTTP or MQTT. By publishing sensor data from devices like Raspberry Pi, Arduino, or NodeMCU to a cloud platform such as ThingSpeak, students gain practical understanding of IoT data transmission, cloud storage, and remote monitoring. The experiment also strengthens knowledge of data logging with timestamps and visualization through cloud dashboards, preparing students to design scalable IoT systems for real-time monitoring and data analytics applications.

Conclusion:

In this experiment, the process of sending sensor data from an IoT device to a cloud platform using communication protocols such as HTTP was studied. Sensor values were transmitted to the ThingSpeak cloud platform and visualized using online dashboards. The experiment provided understanding of IoT data communication, cloud storage, and real-time remote monitoring.

Viva Questions:

1. What is the difference between **HTTP** and **MQTT** protocols in IoT communication?
2. What is **ThingSpeak**, and how is it used in IoT applications?
3. What is an **API key**, and why is it required when sending data to a cloud server?
4. What is meant by a **publisher** in an IoT communication system?
5. Why are **timestamps important** when storing sensor data in cloud platforms?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—10: Integration – Smart Monitoring System

Aim:

a) Combine sensors, actuators, communication protocol, and web/cloud
b) Example: A Smart Weather Station that logs temp/humidity online and triggers fan/LED alerts

Tools Used:

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card with Raspberry Pi OS installed
- Power adapter for Raspberry Pi 4
- HDMI cable and monitor
- USB keyboard and mouse
- DHT11 Temperature and Humidity Sensor
- LED (for alert indication)
- Jumper wires
- Breadboard

Software:

- Raspberry Pi OS
- ThingSpeak Cloud Platform
- Required Python libraries (Adafruit_DHT, requests, RPi.GPIO)

Programming Language: Python

Compiler/IDE: Thonny IDE, Terminal (for running scripts and installing libraries)

Learning Objectives:

1. Understand how different IoT components such as sensors, actuators, and cloud platforms can be integrated into a single system.
2. Collect environmental data such as temperature and humidity using sensors like DHT11.
3. Implement communication between the IoT device and cloud services using internet protocols.
4. Display and monitor sensor data through online dashboards or web interfaces.
5. Develop automated responses such as activating alerts or devices when sensor values exceed predefined thresholds.

Theory:

1. Introduction to Smart Monitoring Systems in IoT

A Smart Monitoring System is an integrated IoT system that continuously observes environmental conditions using sensors and provides real-time feedback or control actions. These systems combine multiple components such as sensors, actuators, communication protocols, and cloud platforms to create intelligent monitoring solutions.

The goal of such systems is to collect data from the environment, process it, and respond automatically when certain conditions occur.

Typical components involved include:

- Sensors for collecting environmental data
- Actuators for performing actions
- Microcontrollers or embedded devices for processing
- Communication protocols for data transmission
- Cloud platforms or web interfaces for monitoring

This experiment demonstrates how these components can be combined to build a simple **Smart Weather Monitoring System**.

2. Components of an IoT Monitoring System

An IoT monitoring system consists of several interconnected components.

Component	Function
Sensor	Collects environmental data
Processing Unit	Processes sensor data
Actuator	Performs control actions
Communication Protocol	Transfers data to remote systems
Cloud Platform	Stores and visualizes data

Conceptual architecture:

```
Sensors → Processing Unit → Communication Protocol → Cloud Platform
                                ↓
                            Actuators
```

This architecture allows the system to both **monitor and respond** to environmental conditions.

3. Sensors for Environmental Monitoring

Sensors are used to measure physical conditions such as temperature, humidity, light intensity, and air quality.

In this experiment, a **DHT11 sensor** is used to measure:

- Temperature
- Relative humidity

Working concept:

```
Environment Conditions
        ↓
      DHT11 Sensor
        ↓
Digital Data Output
        ↓
Raspberry Pi Processing
```

The Raspberry Pi reads the sensor data and processes it for further actions such as cloud uploading or device control.

4. Actuators in Smart Monitoring Systems

Actuators are devices that perform actions based on control signals from the system.

Examples include:

- Fans
- LEDs
- Buzzers
- Motors

In the Smart Weather Monitoring System example:

- **LED** may be used to indicate warning conditions
- **Fan** may be triggered if temperature exceeds a certain threshold

Example logic:

```
If Temperature > Threshold
        ↓
Activate Fan / LED Alert
```

This allows the system to react automatically to environmental changes.

5. Communication Protocol for IoT Data Transfer

To monitor data remotely, sensor readings must be transmitted to a cloud server.

Common IoT communication protocols include:

- HTTP
- MQTT

These protocols allow the device to send data over the internet.

Example communication flow:

```
Sensor Data
      ↓
Raspberry Pi
      ↓
HTTP/MQTT Request
      ↓
Cloud Server (ThingSpeak)
      ↓
Dashboard Visualization
```

This enables users to monitor environmental conditions remotely.

6. Cloud Platform for Data Logging

Cloud platforms store and visualize IoT data collected from devices.

Platforms such as **ThingSpeak** provide features including:

- Data storage
- Graphical data visualization
- Real-time monitoring
- Historical data analysis

Example data stored on cloud platform:

Timestamp	Temperature	Humidity
10:01	26°C	55%
10:02	27°C	56%
10:03	27°C	57%

Graphs generated by the platform help visualize trends over time.

7. Integrated System Workflow

The operation of a Smart Monitoring System follows a sequential process.

```
Read Sensor Data
        ↓
Process Data
        ↓
Send Data to Cloud
        ↓
Check Threshold Conditions
        ↓
Trigger Actuators if Required
```

This integration allows the system to perform both **monitoring and automated control**.

8. Smart Weather Monitoring Example

A Smart Weather Station is a practical application of an integrated IoT monitoring system.

Functions of the system:

- Measure temperature and humidity using DHT11
- Upload data to a cloud platform
- Display readings on a dashboard
- Trigger alerts if environmental conditions exceed thresholds

Example system behavior:

Condition	Action
Temperature > 30°C	Fan ON
High humidity	LED alert
Normal conditions	Monitoring only

Theoretical Steps, Code & Expected Output:

(a) Combine Sensors, Actuators, Communication Protocol, and Web/Cloud (Theoretical)

Steps

1. Connect the **DHT11 temperature and humidity sensor** to the Raspberry Pi GPIO pins.
2. Connect an **LED** to a GPIO pin to act as an alert indicator.
3. Install the required Python libraries.

```
pip3 install Adafruit_DHT
pip3 install requests
```

4. Create a Python program that reads temperature and humidity values from the sensor.
5. Send the sensor data to a cloud platform such as **ThingSpeak** using HTTP requests.
6. Implement control logic to activate an LED if the temperature exceeds a threshold value.

Example System Architecture

```
DHT11 Sensor
      ↓
Raspberry Pi
      ↓
HTTP Request
      ↓
ThingSpeak Cloud
      ↓
Dashboard Visualization
      ↓
Alert LED
```

Example Code

```
import Adafruit_DHT
import requests
import RPi.GPIO as GPIO
import time

sensor = Adafruit_DHT.DHT11
pin = 4

LED = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)

api_key = "YOUR_API_KEY"
url = "https://api.thingspeak.com/update"

while True:
    humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)

    if temperature is not None:
        print("Temperature:", temperature)
        print("Humidity:", humidity)

        payload = {'api_key': api_key, 'field1': temperature, 'field2':
humidity}
        requests.get(url, params=payload)

        if temperature > 30:
            GPIO.output(LED, True)
```

```
        else:
            GPIO.output(LED, False)

    time.sleep(15)
```

Expected Output

Terminal Output:

```
Temperature: 27
Humidity: 60
Temperature: 28
Humidity: 62
```

Cloud Dashboard Example:

Timestamp	Temperature	Humidity
10:01	27°C	60%
10:02	28°C	62%

LED Behavior:

Temperature	LED Status
< 30°C	OFF
> 30°C	ON

(b) Smart Weather Station Example (Logging Data Online and Triggering Alerts)

Working Logic

```
Read Temperature & Humidity
            ↓
Upload Data to Cloud
            ↓
Check Temperature Threshold
            ↓
Trigger LED / Fan Alert
```

Example Automation Logic

```
If Temperature > 30°C
        ↓
Activate LED or Fan
Else
System continues monitoring
```

Expected Cloud Visualization

Example ThingSpeak Graph:

```
Temperature (°C)

32 |        *
30 |      *
28 |    *
26 |  *
   +-------------------
       Time →
```

Humidity Graph:

```
Humidity (%)

70 |        *
65 |      *
60 |    *
55 |  *
   +-------------------
       Time →
```

The dashboard automatically updates whenever new data is uploaded.

Learning Outcomes:

1. Successfully integrate sensors, actuators, and cloud communication into a unified IoT system.
2. Monitor environmental parameters in real time using connected sensors.
3. Upload sensor data to cloud platforms for remote monitoring and visualization.
4. Implement basic automation logic for triggering alerts or devices based on sensor readings.
5. Design simple smart monitoring systems applicable to real-world IoT applications.

Course Outcomes:

This experiment contributes to the attainment of course outcomes by enabling students to design and implement an integrated IoT monitoring system using sensors, actuators, communication protocols, and cloud platforms. By collecting environmental data using sensors such as DHT11, transmitting the data to a cloud platform like ThingSpeak, and triggering alerts or control actions based on threshold conditions, students gain practical experience in end-to-end IoT system development. The experiment strengthens understanding of system integration, real-time data monitoring, cloud connectivity, and automation, which are essential for building scalable and intelligent IoT applications.

Conclusion:

This experiment demonstrated the integration of sensors, actuators, communication protocols, and cloud platforms to develop a simple smart monitoring system. Environmental data collected from sensors was transmitted to a cloud platform for visualization, and automated actions were triggered based on predefined conditions. The experiment highlighted the concept of end-to-end IoT system development and the practical implementation of intelligent monitoring solutions.

Viva Questions:

1. What is meant by a **Smart Monitoring System** in IoT?
2. What components are required to build an integrated IoT system?
3. How does a cloud platform like **ThingSpeak** help in monitoring sensor data?
4. What is the purpose of using **threshold conditions** in IoT automation systems?
5. Explain the working principle of a **Smart Weather Monitoring System**.

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]